



I.E.S. INFANTA ELENA

DAM

Desarrollo de Aplicaciones
Multiplataforma

CICLO FORMATIVO DE GRADO SUPERIOR DESARROLLO DE APLICACIONES WEB



Módulo: DWES – Desarrollo Web en Entorno Servidor

Alumno: Pablo Sevillano Aparicio

Profesor: Sarah González Camacho

Fecha: 13 de febrero de 2026



Sumario

1. Introducción.....	4
1.1 Contexto del proyecto.....	4
Qué es DWES y qué se pretende evaluar.....	4
Qué es la aplicación y qué problema resuelve.....	5
1.2 Objetivos.....	5
Objetivo general.....	5
Objetivos específicos.....	5
1.3 Alcance.....	7
Qué incluye (módulos implementados).....	7
Qué NO incluye (limitaciones declaradas).....	8
Restricciones del entorno y ejecución.....	8
1.4 Planificación y recursos.....	9
1.4.1 Estimación de Recursos y Planificación.....	9
1.4.2 Planificación temporal (Diagrama de Gantt).....	10
1.4.3 Metodología de trabajo (bitácora diaria + sesiones cerradas).....	11
1.4.4 Recursos utilizados.....	12
2. Funcionalidades.....	13
2.1 Funcionalidades de visitante (zona pública).....	13
2.2 Funcionalidades de jugador (zona privada).....	16
2.3 Funcionalidades de administrador.....	21
3. Esquema funcional.....	25
3.1 Flujo global del sistema.....	25
3.2 Flujo jugable “ciclo completo”.....	26
4. Casos de uso.....	27
4.1 Diagrama de casos de uso.....	27
4.2 Descripción textual de casos de uso (tabla o lista).....	29
5. Modelo Entidad–Relación (ER).....	30
5.1 Diagrama ER.....	30
5.2 Explicación del ER.....	31
6. Modelo lógico (relacional).....	32
6.1 Tablas del dominio (RPG).....	32
6.2 Restricciones e integridad.....	33
7. Manual de instalación.....	35
7.1 Requisitos.....	35
7.2 Instalación por clonado.....	35
7.3 Instalación por ZIP.....	36
7.4 Configuración SQLite.....	36
7.5 Migraciones y seeders.....	37
7.6 Assets.....	37
7.7 Arranque del proyecto.....	37
7.8 Reverb / Chat.....	37
7.9 Usuarios de prueba.....	37
8. Manual de usuario.....	39
8.1 Home pública / Lore (visitante).....	39
8.2 Registro.....	40



8.3 Login.....	41
8.4 Home del juego (jugador).....	42
8.5 Creación de personaje + selección de raza.....	43
8.6 Misiones (listado).....	44
8.7 Misión en ejecución (nodo + elecciones).....	45
8.8 Combate PvE (boss).....	46
8.9 Inventario.....	47
8.10 Equipamiento.....	48
8.11 Tienda.....	49
8.12 PvP.....	50
8.13 Chat.....	51
8.14 Páginas legales.....	52
9. Manual de administrador.....	53
9.1 Acceso al panel admin.....	53
9.2 Gestión de bosses (CRUD).....	54
9.3 Gestión de misiones (CRUD).....	55
9.4 Gestión de nodos (CRUD).....	56
9.5 Gestión de elecciones (CRUD + enlaces).....	57
9.6 Publicar/Retirar misión (validación grafo).....	58
9.7 Gestión de ítems/tienda (CRUD).....	59
9.8 Gestión de usuarios/roles/plan.....	60
10. Estructura del proyecto.....	61
10.1 Árbol de directorios (seleccionado y legible).....	61
10.2 Explicación por carpetas.....	65
11. Descripción técnica.....	67
11.1 Arquitectura MVC en Laravel.....	67
11.2 Rutas (web.php / auth.php).....	67
11.3 Controladores (por módulos: Game, Missions, Admin, PvP, Chat).....	67
11.4 Modelos y relaciones (1:1, 1:N, N:M).....	68
11.5 Migraciones y seeders.....	68
11.6 Validación (Form Requests, mensajes, CSRF).....	69
11.7 Seguridad (auth, roles, middleware, sesiones/cookies, CSRF).....	69
11.8 Tiempo real (Chat: eventos, broadcasting, Reverb).....	69
11.9 Comandos artisan usados (migrate, seed, build, limpieza, tiempo real).....	70
12. Conclusión.....	71
12.1 Objetivos alcanzados.....	71
12.2 Dificultades y soluciones.....	71
12.3 Mejoras futuras.....	72
13. Webgrafía.....	73



1. Introducción

El presente proyecto se ha desarrollado como parte del módulo de **Desarrollo Web en Entorno Servidor (DWES)** del ciclo **2º DAW**, con el objetivo de aplicar en un caso real los contenidos trabajados durante el curso: construcción de una aplicación web con lógica en servidor, persistencia de datos y una estructura organizada y mantenible.

La aplicación desarrollada, **El Juramento de Valtherion**, se plantea como un RPG web ejecutable desde navegador. El usuario puede registrarse, iniciar sesión y participar en un ciclo jugable completo: creación de personaje, elección de raza, progresión mediante estadísticas, inventario, equipamiento y tienda, toma de decisiones en misiones ramificadas y combate PvE contra jefes finales. De forma complementaria se incluyen módulos como PvP y chat interno, además de un panel de administración para gestionar el contenido del juego y mantenerlo ampliable sin depender de cambios constantes en el código.

La implementación se ha realizado con **Laravel 12**, siguiendo el patrón **MVC (Modelo–Vista–Controlador)**, y utilizando **SQLite** como sistema de persistencia en formato de fichero portable. El uso de migraciones, seeders, vistas Blade, Vite y, cuando aplica, Reverb, permite un desarrollo estructurado y coherente con el enfoque del módulo, facilitando además la ejecución y revisión del proyecto en un entorno local de evaluación.

1.1 Contexto del proyecto

Qué es DWES y qué se pretende evaluar

El módulo **Desarrollo Web en Entorno Servidor (DWES)** pertenece al ciclo de **2º DAW** y se centra en el diseño y construcción de aplicaciones web donde la lógica principal se ejecuta en el servidor. En este proyecto se pone en práctica lo trabajado durante el curso, especialmente:

- **Programación en PHP** aplicada a un caso real (control de flujo, modularidad y buenas prácticas).
- **Gestión de datos persistentes** mediante un modelo relacional y operaciones completas de lectura/escritura.
- **Arquitectura organizada (MVC)**, separando responsabilidades entre rutas, controladores, modelos y vistas.
- **Validación y seguridad** (sesiones, control de acceso por roles, protección CSRF, validación de formularios y gestión de errores).
- **Documentación técnica y manuales** que permitan ejecutar, comprender y evaluar la aplicación sin depender del autor.



En resumen, se evalúa que la aplicación sea **funcional, coherente, reproducible y documentada**, con un enfoque propio de un desarrollo web de servidor bien estructurado.

Qué es la aplicación y qué problema resuelve

El Juramento de Valtherion es una aplicación web con estética y mecánicas de **RPG** desarrollada para ejecutarse en navegador. La idea principal es ofrecer una experiencia jugable básica y completa (progresión, decisiones, combate y gestión de recursos), junto con un sistema de administración que permita mantener y ampliar el contenido sin modificar directamente la base de datos.

El proyecto resuelve dos necesidades complementarias:

- **Experiencia jugable (jugador):** un usuario puede autenticarse, crear su personaje, tomar decisiones en misiones ramificadas y progresar gestionando inventario, equipo y recursos, culminando en enfrentamientos PvE contra jefes finales y módulos adicionales como PvP y chat.
- **Gestión de contenido (administración):** el contenido del juego (misiones, nodos, elecciones, bosses, ítems y usuarios) se mantiene desde un panel privado, garantizando coherencia y control sobre lo que se publica y se ofrece a los jugadores.

La aplicación se ha desarrollado con **Laravel 12** siguiendo el patrón **MVC**, usando **SQLite** como base de datos (fichero portable) y herramientas propias del ecosistema Laravel (migraciones, seeders, Blade, Vite y, cuando aplica, Reverb para funcionalidades en tiempo real). Esto permite un desarrollo ordenado, mantenible y alineado con los objetivos del módulo.

1.2 Objetivos

Objetivo general

El objetivo general de **El Juramento de Valtherion** es desarrollar una aplicación web completa en entorno servidor que simule una experiencia RPG funcional desde navegador, integrando un ciclo jugable coherente (autenticación, creación de personaje, progresión, misiones, combate y gestión de recursos) junto con un entorno de administración que permita gestionar y ampliar el contenido del juego de forma controlada, manteniendo la coherencia del sistema y aplicando una arquitectura organizada basada en MVC con Laravel 12 y persistencia mediante SQLite.

Objetivos específicos

Objetivos orientados al jugador (usuarios registrados)

- Permitir el **registro e inicio/cierre de sesión** de forma segura.



- Forzar la **creación de personaje** asociada a la cuenta del usuario y mantener la relación 1:1 usuario–personaje.
- Implementar la **selección de raza** con reglas de acceso (según rol/plan) y reparto de puntos/estadísticas conforme a restricciones definidas.
- Ofrecer un sistema de **misiones narrativas ramificadas**, basado en nodos y elecciones, donde cada decisión determine el avance del recorrido.
- Desarrollar un sistema de **combate PvE** contra jefes finales como cierre de misión, con resolución de resultado y entrega de recompensas.
- Incorporar **inventario** para almacenar ítems del jugador, diferenciando equipables y consumibles según corresponda.
- Permitir **equipamiento por slots** y aplicar bonificaciones reales sobre las estadísticas del personaje al equipar objetos.
- Implementar una **tienda interna** donde comprar ítems con rarezas, precios y bonificaciones, conectada al inventario del jugador.
- Incluir un módulo de **chat interno** como herramienta de comunicación dentro del juego (cuando el servicio de tiempo real esté activo).
- Incorporar un módulo **PvP** (jugador contra jugador) como extensión de interacción competitiva dentro del sistema.

Objetivos orientados a la administración (rol administrador)

- Proporcionar un **panel de administración** con acceso restringido por rol.
- Implementar **CRUD completo** para gestionar el contenido principal del juego sin modificar manualmente la base de datos:
 - bosses finales,
 - misiones,
 - nodos y elecciones,
 - ítems/tienda,
 - usuarios y roles/plan.
- Garantizar un **control de publicación** de misiones mediante estados (por ejemplo, draft/published) para evitar que contenido incompleto llegue a los jugadores.
- Integrar mecanismos de **validación de consistencia** del sistema (por ejemplo, coherencia del grafo de misiones y enlaces entre nodos) antes de permitir la publicación.



- Asegurar que el contenido del juego pueda **evolucionar y ampliarse** manteniendo integridad, orden y trazabilidad del sistema.

1.3 Alcance

Qué incluye (módulos implementados)

El Juramento de Valtherion se desarrolla como una aplicación web completa con separación clara entre **zona pública**, **zona privada de juego** y **zona de administración**, integrando los siguientes módulos:

- **Zona pública**
 - Página de inicio / lore del proyecto.
 - Acceso a autenticación (registro e inicio de sesión).
 - Páginas legales (cookies, privacidad y términos), accesibles sin iniciar sesión.
- **Autenticación y control de acceso**
 - Registro e inicio/cierre de sesión.
 - Rutas protegidas por autenticación.
 - Control de permisos mediante roles (jugador / administrador) y plan (si aplica en el sistema).
- **Zona de juego (jugador)**
 - Creación de personaje asociada al usuario (1 usuario ↔ 1 personaje).
 - Selección de raza con reglas de acceso y reparto inicial de estadísticas.
 - Consulta de ficha del personaje y progresión (nivel/experiencia/recursos).
 - **Misiones ramificadas** mediante nodos y elecciones (grafo narrativo).
 - **Combate PvE** contra jefes finales como cierre de misión y obtención de recompensas.
 - **Inventario** del personaje (gestión de ítems y consulta de propiedades).
 - **Equipamiento por slots** con aplicación de bonificaciones reales sobre estadísticas.
 - **Tienda** de ítems con rarezas/bonificaciones y compra mediante moneda interna.



- **Chat interno** como módulo de comunicación dentro del entorno del juego (cuando el servicio en tiempo real está activo).
- **PvP** como módulo adicional de enfrentamiento jugador contra jugador.
- **Zona de administración**
 - Panel privado con acceso restringido a rol administrador.
 - Gestión del contenido del juego mediante CRUD (según módulos del proyecto): bosses, misiones, nodos/elecciones, ítems/tienda y usuarios.
 - Control de publicación de misiones por estados (ej. draft/published) y verificación de coherencia previa a publicación.

Qué NO incluye (limitaciones declaradas)

Para mantener el alcance dentro del periodo de desarrollo y centrarlo en los requisitos del módulo, el proyecto no contempla:

- Despliegue en un entorno de producción externo (servidor real / dominio), quedando la ejecución orientada a entorno local.
- Sistemas avanzados de escalabilidad o alta concurrencia (balanceo, clustering, colas distribuidas).
- Integraciones externas (pasarelas de pago, APIs de terceros o servicios cloud).
- Contenido masivo o extenso (gran cantidad de campañas, bosses o zonas), quedando el sistema preparado para ampliación, pero con contenido base suficiente para demostrar funcionamiento.

Restricciones del entorno y ejecución

El proyecto se ha desarrollado y probado con las siguientes restricciones y condiciones técnicas:

- **Entorno local en Windows** utilizando **XAMPP** y **artisan serve** para ejecución y pruebas.
- **Base de datos SQLite** como almacenamiento principal en formato fichero, lo que aporta portabilidad y simplifica la evaluación sin necesidad de configurar un servidor MySQL.
- **Laravel 12** como framework, con uso de Blade para vistas y Vite para compilación de assets.
- **Reverb / Broadcasting** para funcionalidades en tiempo real (chat), requiriendo el arranque del servicio correspondiente durante la ejecución si se desea probar dicho módulo.



- La interfaz se ha diseñado con enfoque **tipo aplicación**, priorizando una experiencia **sin scroll** en las pantallas principales mediante una estructura por secciones y contenedores ajustados al viewport.
- El diseño es **responsive** y se adapta a **móvil, tablet y escritorio**, manteniendo coherencia visual y navegación estable en los tres formatos.
- Aunque se ejecuta en navegador, la organización de pantallas, componentes y navegación está planteada para comportarse como una **aplicación web** (web app), con módulos accesibles desde menús y vistas independientes.

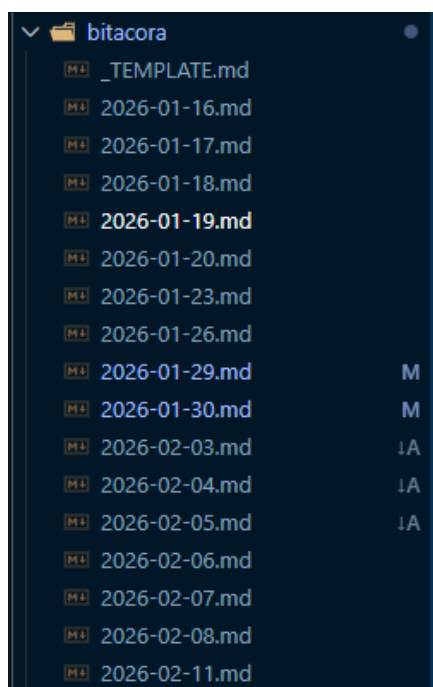
1.4 Planificación y recursos

1.4.1 Estimación de Recursos y Planificación

El desarrollo de *El Juramento de Valtherion* se ha realizado dentro de un periodo acotado, con un planteamiento de trabajo orientado a completar bloques funcionales de manera continua y verificable. La planificación no se ha entendido como un documento teórico separado del desarrollo, sino como un reflejo directo de lo ejecutado en cada sesión, asegurando que tanto el avance técnico como la documentación se mantuvieran alineados durante todo el proyecto.

- Fechas reales (inicio/fin)
 - Fecha de inicio del proyecto: **16/01/2026**
 - Fecha de cierre/entrega: **11/02/2026**Durante este intervalo se fueron cerrando módulos completos de forma progresiva, con revisiones constantes para mantener coherencia entre funcionalidades, datos (SQLite), estructura del proyecto y experiencia de usuario en interfaz responsive.
- Trazabilidad basada en bitácora diaria (/docs/bitacora)

Para garantizar trazabilidad real, se ha llevado un registro diario de actividad en la carpeta **/docs/bitacora**, donde se documentan las tareas realizadas, las decisiones técnicas relevantes y el avance por módulos. Este registro se utiliza como base de planificación “real” porque refleja el trabajo efectivo ejecutado en cada fecha, y permite justificar el orden de construcción del proyecto, los cambios aplicados y las validaciones hechas en cada tramo.



***Ilustración: Carpeta /docs/bitacora (listado de archivos por fecha)*

1.4.2 Planificación temporal (Diagrama de Gantt)

Para representar de forma visual la distribución del trabajo, se incluye un **Diagrama de Gantt** donde se agrupan las tareas por módulos y por semanas, reflejando cómo se fue construyendo la aplicación desde la base (estructura Laravel, autenticación, persistencia en SQLite) hasta los sistemas de juego (personaje, razas, misiones, combate, inventario, tienda) y el cierre final del proyecto (revisión general, ajustes y documentación).

- Distribución del trabajo por módulos y semanas
El diagrama permite identificar qué bloques se abordaron en cada tramo del calendario: primero la base del proyecto y el modelo de datos, después los módulos jugables principales y, en paralelo, la consolidación de panel de administración y contenido gestionable. Esta vista temporal ayuda a entender que el proyecto se desarrolló con una progresión lógica: primero estructura y reglas, después funcionalidades completas, y finalmente consolidación y verificación.
- Documentación + cierre técnico como tareas transversales
A lo largo del desarrollo, la documentación y el cierre técnico se mantuvieron como tareas transversales: no se dejaron para el final, sino que se fueron actualizando conforme se implementaban módulos. De la misma forma, se realizaron revisiones de coherencia (funcionalidad, navegación, consistencia de datos y control de acceso) como parte del proceso, no como un añadido de última hora.

**Diagrama de Gantt — El Juramento de Valtherion (16/01/2026-11/02/2026)**

Módulo / Fase	16/01-18/01	19/01-25/01	26/01-01/02	02/02-08/02	09/02-11/02
Memoria (documentación)					
Arranque y alcance					
Análisis del sistema					
Casos de uso y permisos					
MER / Modelo lógico BBDD					
Diseño UI/UX y prototipos					
Base y configuración Laravel					
Autenticación y perfiles					
Juego (UI + stats)					
Misiones (grafo + lógica)					
Combate PvE / Bosses					
Inventario + Equipamiento					
Tienda + Items					
Panel Admin					
PvP					
Legal / RGPD (cookies, privacy, terms)					
QA / pruebas / cierre (cache, builds, fixes, limpieza)					

**Ilustración: Diagrama de Gantt

1.4.3 Metodología de trabajo (bitácora diaria + sesiones cerradas)

La metodología aplicada se basó en un enfoque de trabajo por sesiones completas, buscando que cada jornada dejara un avance tangible y verificable dentro del proyecto. Esta forma de trabajo facilita mantener el control del estado del sistema, reduce acumulación de tareas abiertas y permite comprobar de forma más clara que cada parte del proyecto funciona antes de continuar con el siguiente bloque.

- **Bitácora diaria como registro oficial**
La bitácora diaria actúa como el registro principal del desarrollo, documentando qué se implementó, qué ajustes se realizaron y qué decisiones se tomaron. Además de aportar trazabilidad, sirve como soporte para justificar el proceso seguido y para localizar rápidamente cambios o hitos concretos del proyecto si fuese necesario revisarlos durante la evaluación.
- **Sesiones cerradas (tarea iniciada y cerrada en la misma jornada cuando es posible)**
El proyecto se organizó en “sesiones cerradas”: se inicia una tarea, se implementa, se valida en interfaz y se deja terminada dentro del mismo día siempre que es posible. Este enfoque ayuda a mantener la coherencia general del sistema y evita que queden funcionalidades a medias o sin validar, especialmente en módulos con dependencias entre sí (por ejemplo: misiones → combate → recompensas → inventario/tienda).

- Actualización continua de la memoria durante el desarrollo

La memoria se fue actualizando durante el desarrollo, evitando redactarla al final de golpe. Con esto se asegura que la documentación describe lo que realmente se ha construido y cómo se ha construido, manteniendo coherencia con la evolución del proyecto, los módulos implementados y el orden real de trabajo reflejado en bitácora.

1.4.4 Recursos utilizados

Los recursos utilizados se han orientado a cubrir el ciclo completo de desarrollo del proyecto: implementación backend con Laravel, gestión de assets frontend, persistencia con SQLite, control de versiones, ejecución local y validación funcional en navegador. Al tratarse de un proyecto evaluable en entorno local, se priorizó un stack reproducible y portable, evitando dependencias externas innecesarias.

Software

Se han utilizado herramientas estándar del ecosistema Laravel y del desarrollo web moderno para construir una aplicación completa y mantenible:

- **Laravel 12** como framework principal, siguiendo arquitectura MVC.
- **Composer** para instalación y gestión de dependencias del backend.
- **Node.js + NPM** para dependencias frontend.
- **Vite** para compilación de assets (build y modo desarrollo).
- **Blade** como motor de plantillas para las vistas del proyecto.
- **Tailwind CSS** para maquetación y estilos responsive de la interfaz.
- **SQLite** como base de datos en formato fichero portable.
- **Git / GitHub** para control de versiones, histórico de cambios y trazabilidad.
- **Visual Studio Code** como IDE principal de desarrollo.
- **XAMPP** como entorno local en Windows para soporte de ejecución con PHP/servidor local.

Hardware

La ejecución del proyecto se realizó en un entorno de trabajo local con un equipo Windows con capacidad suficiente para:

- ejecutar servidor local (PHP / Artisan),
- compilar assets (Vite),
- gestionar persistencia con SQLite,
- realizar pruebas funcionales en un navegador moderno.
- Para las validaciones y revisión de interfaz se utilizaron navegadores actuales (Chrome/Brave/Firefox), verificando la experiencia responsive en móvil, tablet y escritorio.

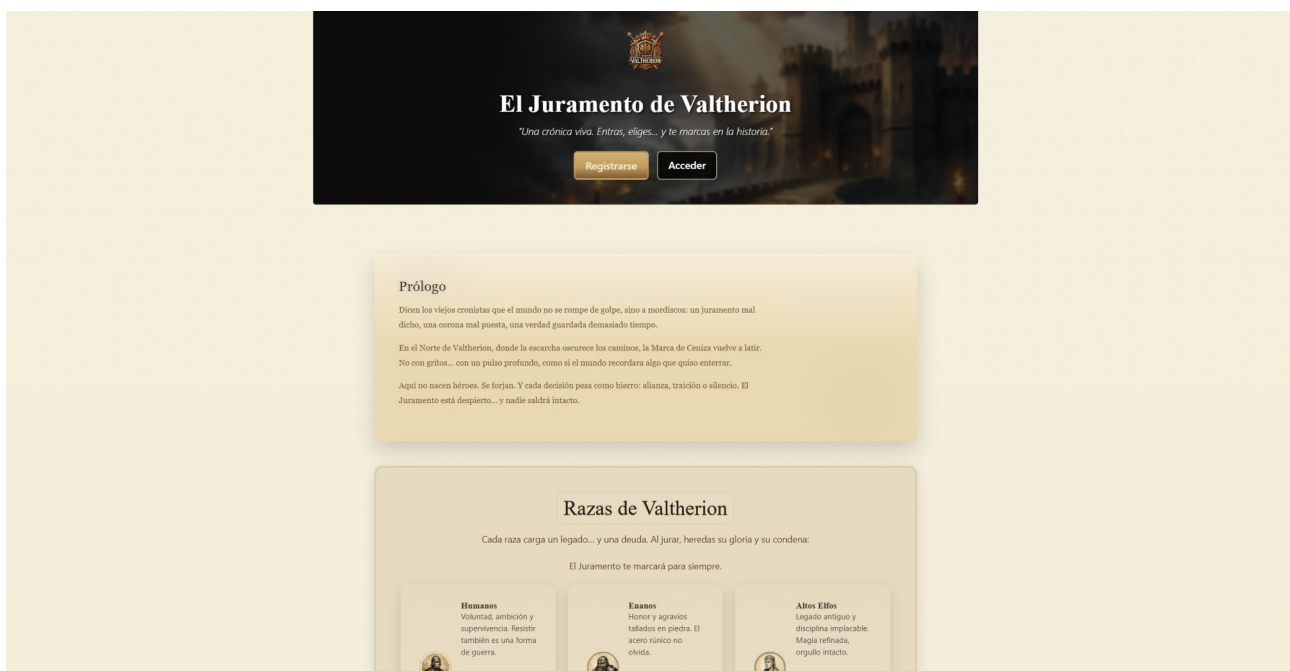
2. Funcionalidades

2.1 Funcionalidades de visitante (zona pública)

La zona pública corresponde a la parte accesible sin iniciar sesión. Su objetivo es presentar el proyecto, ofrecer un punto de entrada claro a la autenticación y garantizar el acceso a la información legal requerida. Esta zona actúa como “puerta” del sistema: permite entender qué es la aplicación antes de entrar a la parte jugable y mantiene separadas, desde el primer momento, las rutas públicas de las rutas protegidas.

- Home:

La página inicial funciona como presentación del universo del juego y del propósito general del proyecto. Desde aquí el visitante puede leer el contexto narrativo (lore) y acceder a los enlaces principales de navegación pública. Esta pantalla está diseñada para ser clara y directa: explica qué ofrece la aplicación y guía al usuario hacia el registro o el inicio de sesión sin necesidad de conocer previamente el sistema.



***Ilustración: Home pública*

-Páginas legales:

La aplicación incluye páginas legales accesibles desde la zona pública, normalmente enlazadas desde la navegación o el footer. Estas secciones aportan transparencia y completan el sitio con contenido obligatorio o recomendado para un proyecto web evaluable, permitiendo que cualquier visitante consulte la información sin necesidad de autenticación.

Legales

Consulta nuestras políticas y condiciones.

[Política de cookies](#)

[Terminos y condiciones](#)

[Política de privacidad](#)

Terminos y condiciones

Al registrarte aceptas estas condiciones de uso.

Uso de la cuenta

Tu cuenta es personal. Eres responsable del uso de tus credenciales.

Conducta

No se permite contenido ofensivo, abuso o uso indebido de la plataforma.

Cambios

Podemos actualizar estos terminos cuando sea necesario.

Política de cookies

Usamos cookies para permitir el acceso, el registro y el funcionamiento básico del sitio.

Que cookies usamos

- Cookies técnicas de sesión para mantener el acceso.
- Cookies de consentimiento para recordar tu elección.

Como puedes gestionarlas

Puedes borrar o bloquear cookies desde la configuración de tu navegador. Si las bloqueas, el registro y el acceso pueden quedar deshabilitados.

Política de privacidad

Solo almacenamos los datos necesarios para el funcionamiento del juego.

Datos recogidos

- Nombre de usuario y correo.
- Datos de progreso de juego.

Derechos

Puedes solicitar la modificación o eliminación de tus datos.

****Ilustración: Páginas legales (Cookies / Privacidad / Términos)**



-Registro / Login:

Desde la zona pública el visitante puede registrarse para crear una cuenta o iniciar sesión si ya dispone de credenciales. Esta funcionalidad es el punto de transición entre la zona pública y la zona privada del juego: una vez autenticado, el usuario pasa a rutas protegidas y se habilita el flujo jugable. El sistema valida los datos introducidos y gestiona la sesión de forma segura, separando claramente lo que puede hacer un visitante de lo que puede hacer un usuario autenticado.



Nombre

Ej: Valtherion

Correo Electrónico

Ej: usuario@dominio.com

Contraseña

Mínimo 8 caracteres

Confirmar Contraseña

Repite la contraseña

☐ Acepto el uso de cookies. [Ver cookies](#)

☐ Acepto los términos y condiciones. [Ver términos](#)

☐ ¿Ya estás registrado?

REGISTRARSE



Correo Electrónico

Contraseña

☐ Recordarme

☐ Acepto el uso de cookies. [Ver cookies](#)

☐ Acepto los términos y condiciones. [Ver términos](#)

☐ ¿Olvidaste tu contraseña? [¿No tienes cuenta? Regístrate](#)

INICIAR SESIÓN

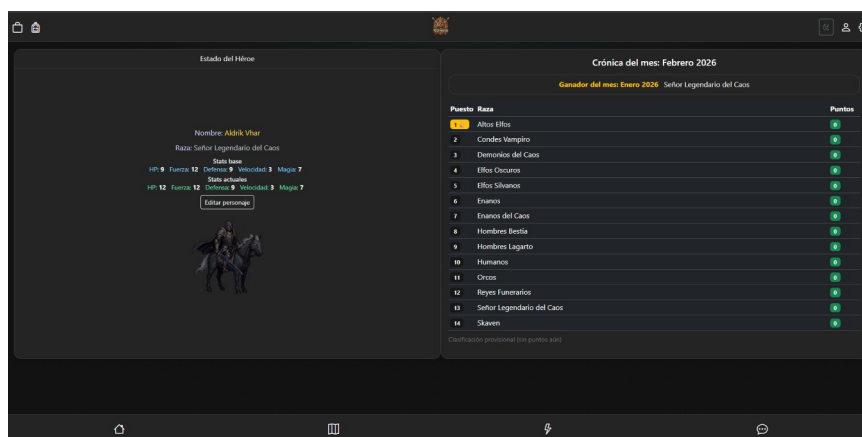
****Ilustración: Formularios de Registro e Inicio de sesión**

2.2 Funcionalidades de jugador (zona privada)

La zona privada corresponde a la parte jugable de la aplicación y solo está disponible tras iniciar sesión. En este entorno el usuario deja de ser un visitante y pasa a interactuar con el sistema como jugador, accediendo a un ciclo completo de juego: creación de personaje, progresión, toma de decisiones, combate y gestión de recursos. Toda esta zona está protegida por autenticación y organizada por módulos para mantener una navegación clara y coherente, con un enfoque tipo aplicación (vistas ajustadas al viewport, evitando scroll innecesario en las pantallas principales y manteniendo una experiencia estable en móvil, tablet y escritorio).

-Home del juego (panel del jugador):

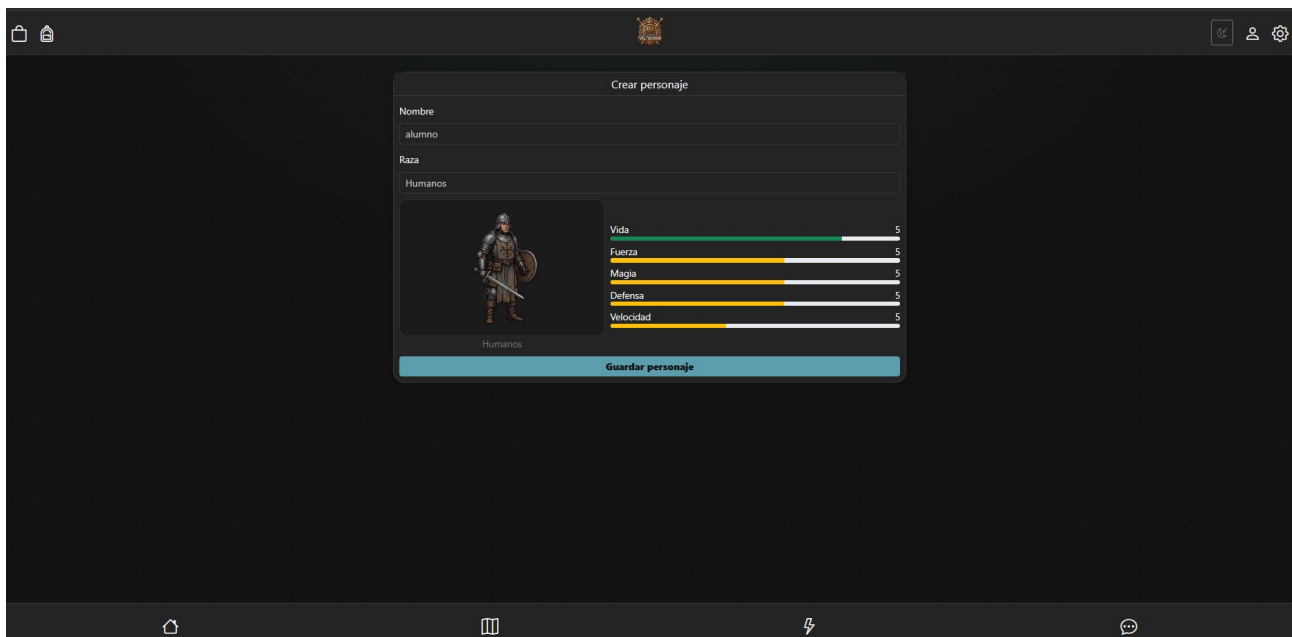
Tras iniciar sesión, el usuario accede al panel principal del juego, que actúa como centro de navegación hacia todos los módulos. Desde aquí se muestran datos básicos del personaje (si existe) y accesos directos a misiones, combate, inventario, equipamiento, tienda, PvP y chat. Este panel mantiene una estructura tipo aplicación, con vistas ajustadas al viewport y navegación consistente en móvil, tablet y escritorio.



***Ilustración: Home del juego*

-Creación de personaje:

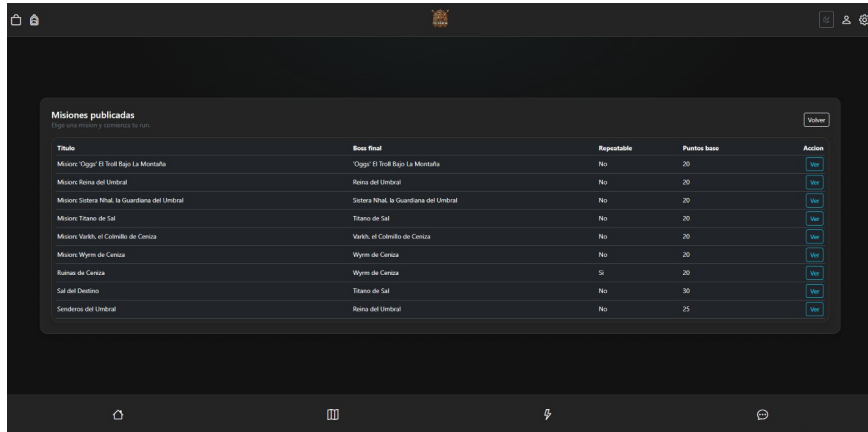
En el primer acceso a la zona privada, el sistema fuerza la creación de un personaje asociado a la cuenta. Este paso es obligatorio porque el personaje representa el estado jugable: estadísticas, progreso, inventario y recursos. La aplicación garantiza la relación 1:1 entre usuario y personaje, evitando inconsistencias (un usuario no puede crear más de un personaje dentro del mismo perfil).



***Ilustración: Creación de personaje*

-Misiones ramificadas (grafo):

El núcleo narrativo del proyecto se basa en misiones estructuradas como un grafo de nodos y elecciones. El jugador inicia una misión, lee el contexto del nodo actual y elige entre opciones que determinan el avance hacia otros nodos o hacia el tramo final. Esta estructura permite rutas distintas dentro de una misma misión, manteniendo el sistema preparado para ampliar contenido sin rehacer lógica base.

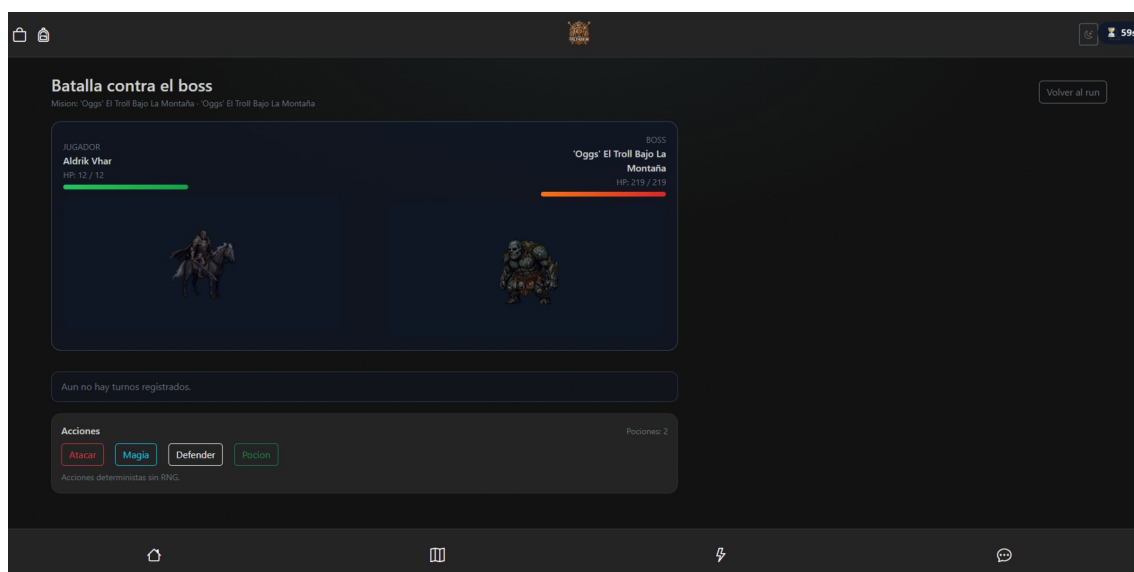


Título	Boss final	Repeatable	Puntos base	Acción
Misión: 'Oggs' El Troll Bajo La Montaña	'Oggs' El Troll Bajo La Montaña	No	20	Ver
Misión: Reina del Umbral	Reina del Umbral	No	20	Ver
Misión: Sistema Nhal, la Guardiana del Umbral	Sistema Nhal, la Guardiana del Umbral	No	20	Ver
Misión: Titano de Sal	Titano de Sal	No	20	Ver
Misión: Varkh, el Colmillo de Ceniza	Varkh, el Colmillo de Ceniza	No	20	Ver
Misión: Wyrm de Ceniza	Wyrm de Ceniza	No	20	Ver
Misión: Wyrm de Ceniza	Wyrm de Ceniza	Si	20	Ver
Sal del Destino	Titano de Sal	No	30	Ver
Senderos del Umbral	Reina del Umbral	No	25	Ver

***Ilustración: Ejecución de misión (nodo + elecciones)*

-Combate PvE (boss final):

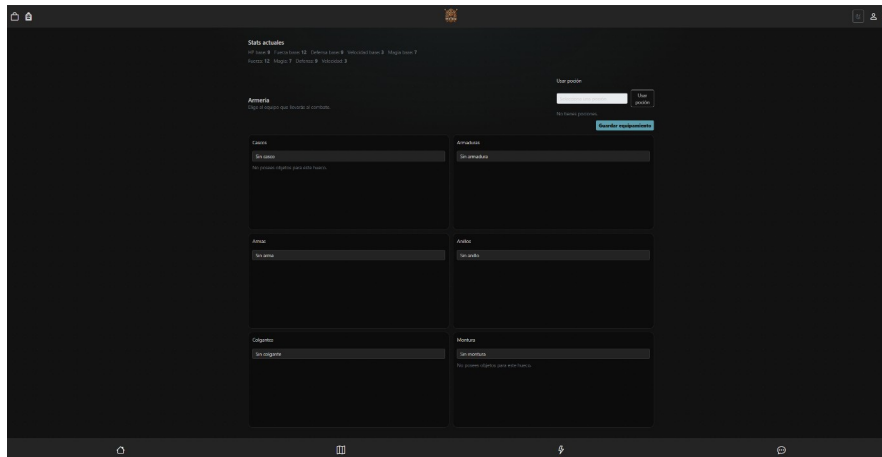
Como cierre del recorrido de misión (o cuando se alcanza el nodo final correspondiente), se desencadena un combate PvE contra un boss final configurado en el sistema. El combate se gestiona dentro del ciclo jugable: se muestran valores relevantes, se ejecutan acciones según la lógica implementada y se determina un resultado. Tras la victoria, el jugador recibe recompensas asociadas (por ejemplo oro/experiencia y/o recompensas definidas por misión).



***Ilustración: Combate PvE contra boss*

-Inventario + consumibles:

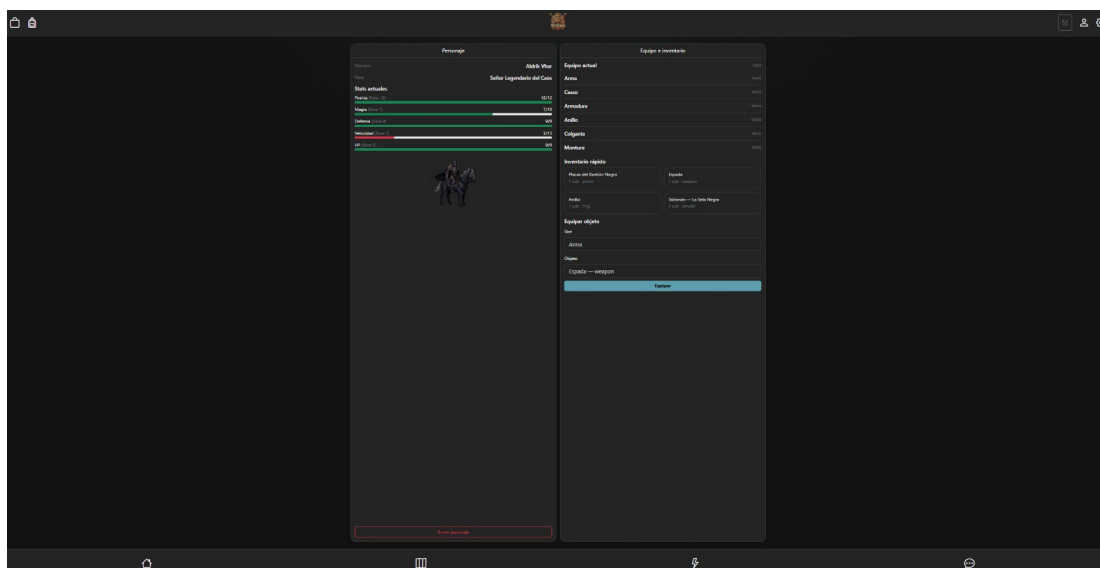
El jugador dispone de un inventario donde se almacenan los objetos obtenidos o comprados. El inventario diferencia entre ítems equipables y consumibles, mostrando sus propiedades (rareza, descripción, bonificaciones o efectos). Este módulo funciona como el almacén principal del personaje y como punto de conexión entre recompensas, tienda y equipamiento.



***Ilustración: Inventario del jugador*

-Equipamiento por slots:

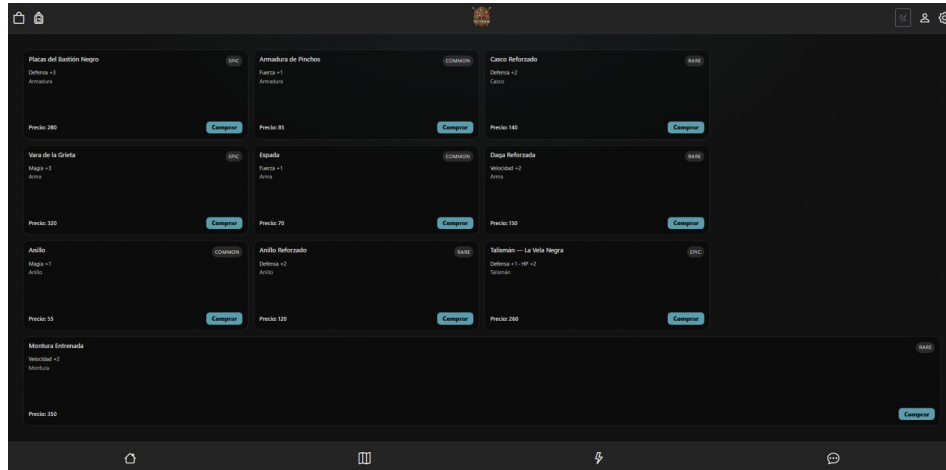
El sistema permite equipar ítems en ranuras concretas (slots) del personaje. Al equipar, las estadísticas finales del jugador se ven modificadas por las bonificaciones del objeto, manteniendo una progresión real basada en el equipo. El sistema controla compatibilidad por slot y mantiene un estado consistente del personaje, permitiendo sustituir piezas y reflejar cambios de forma inmediata dentro de la experiencia.



***Ilustración: Equipamiento por slots*

-Tienda:

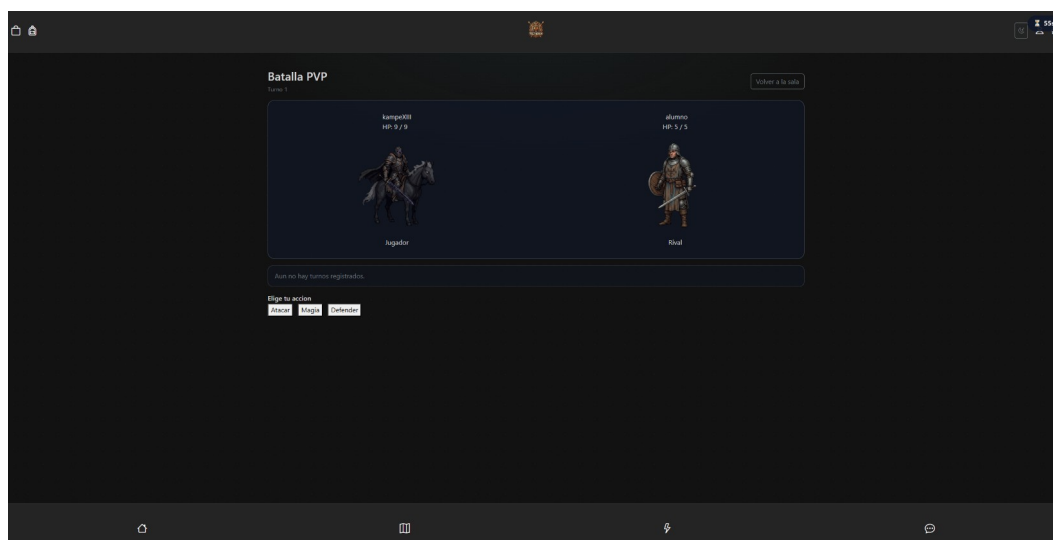
La tienda ofrece un catálogo de ítems con precios, rarezas y bonificaciones. El jugador puede comprar usando la moneda interna y, tras la compra, el ítem pasa al inventario. Este módulo está diseñado como parte de la progresión: conecta la economía del juego con la mejora de estadísticas mediante equipamiento o el uso de consumibles.



***Ilustración: Tienda de ítems*

-PvP:

El módulo PvP amplía la experiencia permitiendo enfrentamientos entre jugadores. Su objetivo es reutilizar el concepto central del proyecto (estadísticas + equipo + progresión) dentro de un contexto jugador contra jugador. El módulo PvP permite acceder a un lobby/sala, seleccionar rival y resolver el combate en función de las estadísticas y el equipamiento del personaje, manteniendo siempre acceso desde la zona privada y coherencia con el resto de módulos.

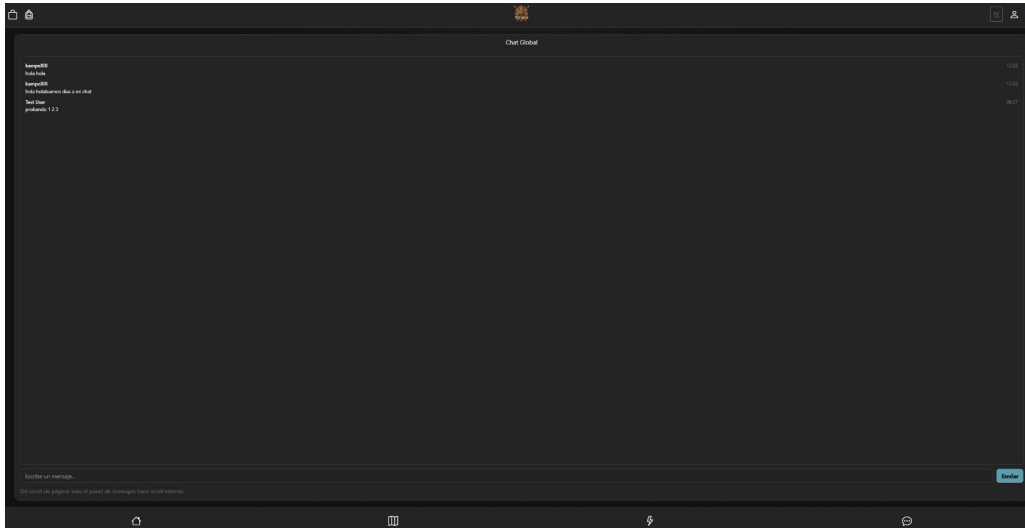


***Ilustración: PvP (lobby/sala/combate)*



-Chat (Reverb):

La zona privada incluye un chat interno para comunicación dentro del juego. Este módulo se apoya en broadcasting y Reverb cuando el servicio está activo, permitiendo envío y recepción de mensajes en tiempo real. Forma parte de los módulos “sociales” del sistema y refuerza la sensación de entorno vivo dentro de la aplicación.

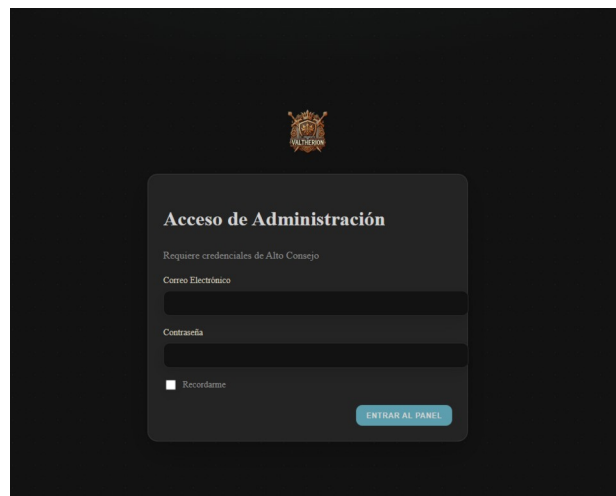


***Ilustración: Chat interno (Reverb)*

2.3 Funcionalidades de administrador

-Acceso a /admin:

El sistema incluye una zona de administración accesible únicamente por usuarios con rol administrador. Desde esta ruta se centraliza la gestión completa del contenido del juego y se separa de forma estricta la experiencia del jugador, asegurando que las acciones administrativas no estén disponibles en la zona privada estándar.



***Ilustración: Acceso al panel /admin*



-CRUD bosses:

Desde el panel se gestiona el catálogo de bosses finales. Se pueden crear, editar y eliminar bosses, manteniendo sus atributos y recursos asociados para que puedan vincularse correctamente a las misiones y al combate PvE.

Bosses finales						
Gestiona los bosses finales y sus datos base.						
Nombre	Slug	HP	Danio	Defensa	Acciones	
'Oggi' El Troll Bajo La Montaña	oggi-el-troll-bajo-la-montana	420	38	22	Editar	Borrar
Reina del Umbral	reina-del-umbral	1100	140	50	Editar	Borrar
Sistara Nhul, la Guardiana del Umbral	sistara-nhul-la-guardiana-del-umbral	560	24	35	Editar	Borrar
Titano de Sal	titano-de-sal	1600	95	90	Editar	Borrar
Varth, el Colmillo de Ceniza	varth-el-colmillo-de-ceniza	352	52	14	Editar	Borrar
Wyrm de Ceniza	wyrm-de-ceniza	1200	120	60	Editar	Borrar

***Ilustración: CRUD de bosses*

-CRUD misiones:

El administrador puede crear, editar y eliminar misiones, definiendo su información general y controlando su estado. La misión actúa como contenedor principal del contenido narrativo y del recorrido jugable, incluyendo su configuración para el tramo final y recompensas asociadas.

Misiones						
Gestiona las misiones narrativas base.						
Título	Slug	Estado	Repetible	Puntos base	Boss final	Acciones
Misión Sistara Nhul, la Guardiana del Umbral	misión-sistara-nhul-la-guardiana-del-umbral	published	No	20	Sistara Nhul, la Guardiana del Umbral	Nuevos Editar Borrar
Misión Wyrm de Ceniza	misión-wyrm-de-ceniza	published	No	20	Wyrm de Ceniza	Nuevos Editar Borrar
Misión Titano de Sal	misión-titano-de-sal	published	No	20	Titano de Sal	Nuevos Editar Borrar
Misión Reina del Umbral	misión-reina-del-umbral	published	No	20	Reina del Umbral	Nuevos Editar Borrar
Misión 'Oggi' El Troll Bajo La Montaña	misión-oggi-el-troll-bajo-la-montana	published	No	20	'Oggi' El Troll Bajo La Montaña	Nuevos Editar Borrar
Misión Varth, el Colmillo de Ceniza	misión-varth-el-colmillo-de-ceniza	published	No	20	Varth, el Colmillo de Ceniza	Nuevos Editar Borrar
Senderos del Umbral	senderos-del-umbral	published	No	25	Reina del Umbral	Nuevos Editar Borrar
Ruinas de Ceniza	ruinas-de-ceniza	published	Si	20	Wyrm de Ceniza	Nuevos Editar Borrar
Sal del Destino	sal-del-destino	published	No	30	Titano de Sal	Nuevos Editar Borrar

***Ilustración: CRUD de misiones*



-CRUD nodos/elecciones:

Cada misión se construye internamente mediante nodos narrativos y elecciones. El panel permite gestionar estos elementos: crear nodos, editar textos, y definir elecciones con su dificultad, resultado y enlace al siguiente nodo o transición al final. Esto garantiza que el contenido ramificado se mantenga ordenado y editable sin tocar código.

Paso	Título	Inicio	Opciones	Acciones
Paso 1	Entrada del tramo	-	3	Editar, Opciones, Borrar
Paso 2	Ruta del norte	-	3	Editar, Opciones, Borrar
Paso 2	Ruta del sur	-	3	Editar, Opciones, Borrar
Paso 3	Cuena velado	-	3	Editar, Opciones, Borrar
Paso 3	Paseo estrecho	-	3	Editar, Opciones, Borrar
Paso 4	Puerto avanzado	-	3	Editar, Opciones, Borrar
Paso 4	Paseo rota	-	3	Editar, Opciones, Borrar
Paso 5	Camara de espera	-	3	Editar, Opciones, Borrar
Paso 6	Umbral del bos	-	3	Editar, Opciones, Borrar

***Ilustración: Gestión de nodos y elecciones (admin)*

-Publicación (draft/published + validación grafo):

Las misiones se controlan por estados para evitar que contenido incompleto o incoherente llegue a los jugadores. Antes de publicar, se comprueba la consistencia del grafo: existencia de inicio, enlaces válidos entre nodos, rutas coherentes y final correctamente conectado. Solo cuando la misión es válida se habilita su publicación.

Título	Slug	Estado	Repetable	Puntos base	Rewards
Misión: Sotera Nihul, la Guardiara del Umbral	mision-sotera-nihul-la-guardiana-del-umbral	Published	Se puede repetir	20	XP: 120, Oro: 8, Items: 3

***Ilustración: Publicación de misión y validación de grafo*



-CRUD ítems/tienda:

El panel permite gestionar el catálogo de ítems disponibles en el sistema: creación, edición y eliminación. Se controlan propiedades como tipo, slot, rareza, bonificaciones y precio, asegurando coherencia con el inventario, el equipamiento y la progresión del juego.

Montura en venta

Semana 7, 2026

Montura de la semana

Selecciona una montura

Guardar montura

Montura actual: Montura Entrenada

***Ilustración: CRUD de ítems / tienda*

-Gestión de usuarios/roles/plan:

El administrador puede consultar usuarios y gestionar los permisos del sistema mediante roles y plan. Esto permite controlar el acceso al panel admin y a contenidos restringidos dentro del juego, manteniendo una separación clara entre jugador estándar, premium y administrador.

Usuarios

Buscar por ID, nombre o correo y gestionar usuarios.

Volver al panel

ID	Nombre	Correo	Rol	Plan	Acciones
5	alumno	alumno@alumno.com	user	free	Editar Borrar
3	Sany04	sagratme@gmail.com	user	free	Editar Borrar
2	kamp008	pablosevillano90@gmail.com	admin	premium	Editar Borrar
1	Test User	test@example.com	user	free	Editar Borrar

Editar usuario

Selecciona crear o editar un usuario de la lista.

Buscar y gestionar

Buscar por ID, nombre o correo

Crear usuario

***Ilustración: Gestión de usuarios, roles y plan*



3. Esquema funcional

3.1 Flujo global del sistema

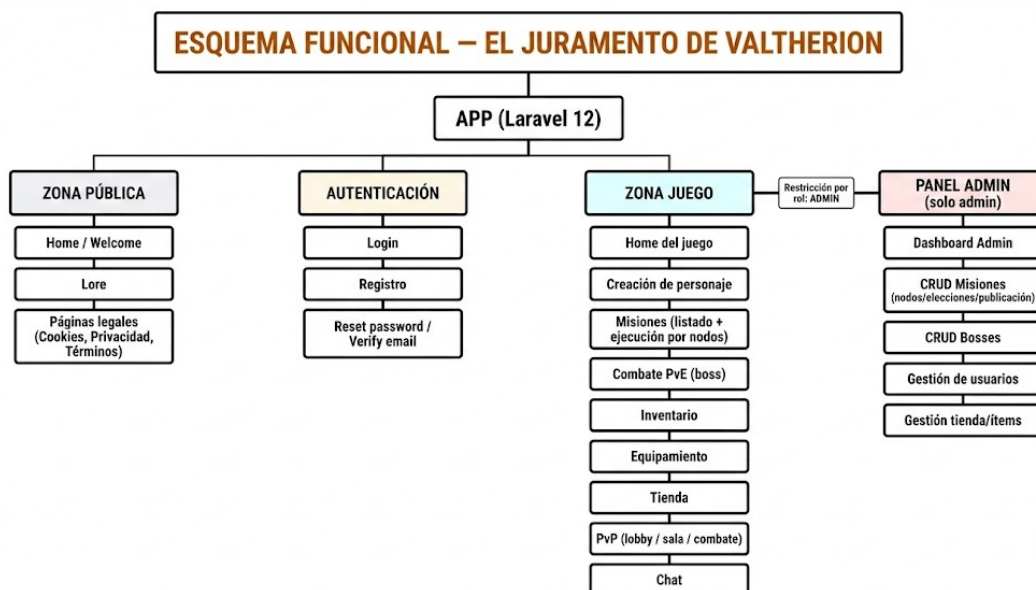
El esquema funcional resume cómo se organiza la aplicación por zonas y cómo fluye el usuario entre ellas. El punto de entrada es la zona pública, desde la cual se accede a autenticación. Una vez el usuario inicia sesión, el sistema conduce a la zona privada del juego y, si el usuario tiene rol administrador, habilita además el acceso a la zona de administración. Esta separación permite mantener una estructura clara y controlada: lo público presenta y contextualiza el proyecto, lo privado ofrece la experiencia jugable y lo administrativo centraliza la gestión de contenido.

Flujo general:

- Zona pública → autenticación → creación de personaje (si no existe) → home del juego → módulos (misiones, combate, inventario, equipamiento, tienda, PvP, chat) → admin (solo si rol)

A nivel modular, la aplicación se divide en:

- Zona pública: presentación/lore + acceso a registro/login + páginas legales
- Autenticación: control de acceso a la zona privada
- Zona de juego: panel principal y navegación entre módulos jugables
- Zona admin: gestión del contenido del juego (misiones, bosses, ítems, usuarios) con acceso restringido



***Ilustración: Esquema funcional*



3.2 Flujo jugable “ciclo completo”

El ciclo jugable describe la secuencia completa que recorre un jugador desde que entra al sistema hasta que progresa y vuelve a prepararse para la siguiente misión. Este flujo es la base de la experiencia RPG: el jugador entra, consolida su estado (personaje), toma decisiones narrativas, resuelve el combate final y obtiene recompensas que impactan en su progresión (oro, experiencia e ítems). Después, utiliza tienda e inventario/equipamiento para mejorar su personaje y estar listo para repetir el ciclo con nuevas misiones.

Flujo jugable completo:

- Login → personaje → misión → decisiones → boss → recompensa → tienda/equipo → siguiente misión

Este ciclo conecta los módulos principales y garantiza que el jugador siempre tenga una ruta clara de progresión:

- Misiones aportan narrativa y decisiones
- Combate aporta resolución y reto
- Recompensas alimentan progresión (recursos y mejoras)
- Tienda + inventario/equipamiento convierten recursos en mejoras reales de estadísticas



4. Casos de uso

4.1 Diagrama de casos de uso

El diagrama de casos de uso representa, de forma clara y académica, las funcionalidades principales del sistema desde la perspectiva de los distintos tipos de usuario que interactúan con la aplicación. Se distinguen tres actores (Visitante, Jugador y Administrador) y se agrupan los casos de uso según el acceso real de cada uno, mostrando de manera visual qué acciones están disponibles en la zona pública, cuáles pertenecen a la zona privada jugable y cuáles quedan reservadas al panel de administración.

Actores del sistema:

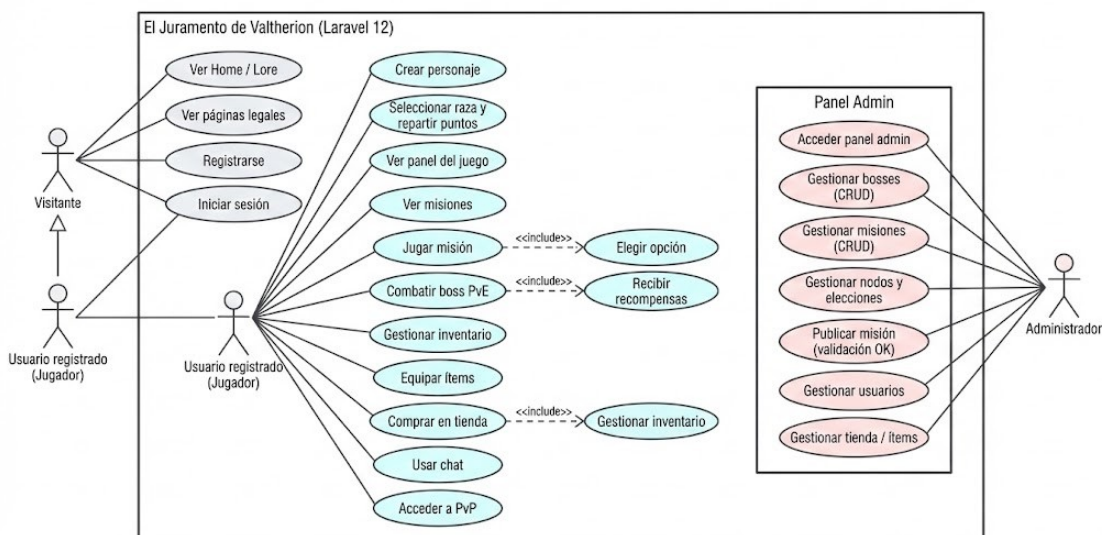
- Visitante: usuario sin autenticación, con acceso a la información pública del proyecto y a la autenticación.
- Jugador: usuario autenticado, con acceso completo a la experiencia jugable (personaje, misiones, combate, progresión y módulos internos).
- Administrador: usuario con rol de administración, con acceso al panel admin y a la gestión de contenido.

Casos de uso agrupados por actor:

- Visitante:
 - Ver Home / Lore
 - Ver páginas legales
 - Registrarse
 - Iniciar sesión
- Jugador:
 - Crear personaje
 - Seleccionar raza y repartir puntos
 - Ver panel del juego
 - Ver misiones
 - Jugar misión (por nodos y elecciones)
 - Elegir opción
 - Combatir boss PvE

- Recibir recompensas
 - Gestionar inventario
 - Equipar ítems
 - Comprar en tienda
 - Usar chat
 - Acceder a PvP
- Administrador:
 - Acceder panel admin
 - Gestionar bosses (CRUD)
 - Gestionar misiones (CRUD)
 - Gestionar nodos y elecciones
 - Publicar misión (validación OK)
 - Gestionar usuarios
 - Gestionar tienda / ítems

CASOS DE USO — EL JURAMENTO DE VALTHERION



**Ilustración: Diagrama de casos de uso



4.2 Descripción textual de casos de uso (tabla o lista)

La descripción textual complementa el diagrama y aporta el detalle operativo de cada funcionalidad. En esta sección se documentan los casos de uso relevantes con un formato homogéneo, de forma que se pueda entender qué ocurre antes, durante y después de cada acción, incluyendo comportamientos esperados, validaciones y salidas alternativas ante errores. Esto permite que la memoria no se limite a “lo visual”, sino que explique el funcionamiento real del sistema desde el punto de vista del usuario y de la lógica del servidor.

Para cada caso de uso relevante se describe:

- Actor: quién ejecuta la acción (Visitante, Jugador o Administrador).
- Precondiciones: requisitos necesarios antes de poder ejecutar el caso (por ejemplo, estar autenticado, tener personaje creado o tener rol admin).
- Flujo principal: pasos en orden que representan el funcionamiento normal.
- Flujos alternativos: respuestas del sistema ante situaciones típicas (validación fallida, acceso no autorizado, datos incompletos, recursos inexistentes).
- Postcondiciones: estado final tras ejecutar el caso (cambios persistentes, navegación resultante, datos actualizados, recompensas registradas, etc.).

Estructura utilizada para documentarlos :

- Nombre del caso de uso
- Actor
- Precondiciones
- Flujo principal
- Flujos alternativos
- Postcondiciones

5. Modelo Entidad–Relación (ER)

5.1 Diagrama ER

El diagrama Entidad–Relación (ER) representa la estructura conceptual de datos del sistema, centrada únicamente en el dominio RPG de El Juramento de Valtherion. Es decir, se modelan las entidades propias del juego y sus relaciones reales (personaje, razas, misiones, nodos, elecciones, ítems, bosses, etc.), evitando mezclar tablas internas del framework que no aportan información funcional al análisis del dominio.

Este diagrama permite ver de un solo vistazo cómo se sostiene el estado jugable: el usuario se relaciona con su personaje, el personaje acumula inventario, puede equipar ítems con efecto real sobre estadísticas y progresa a través de misiones ramificadas que se resuelven mediante un grafo de nodos y elecciones, culminando en combates PvE contra bosses finales.

Puntos clave representados en el ER:

- Entidades del dominio RPG:
 - Usuario (como propietario del progreso)
 - Personaje
 - Raza
 - Ítem
 - Inventario (relación personaje–ítems)
 - Equipamiento (slots + ítems equipados)
 - Misión
 - Nodo de misión
 - Elección (choice) y enlace entre nodos
 - Recompensas
 - Boss final / combate PvE
- Relaciones estructurales:
 - 1:1 usuario–personaje como base de consistencia del progreso
 - N:M personaje–ítems para inventario y compras
 - Misiones como grafo: misión → nodos → elecciones → siguiente nodo
 - Boss vinculado al cierre de misión y a la entrega de recompensas



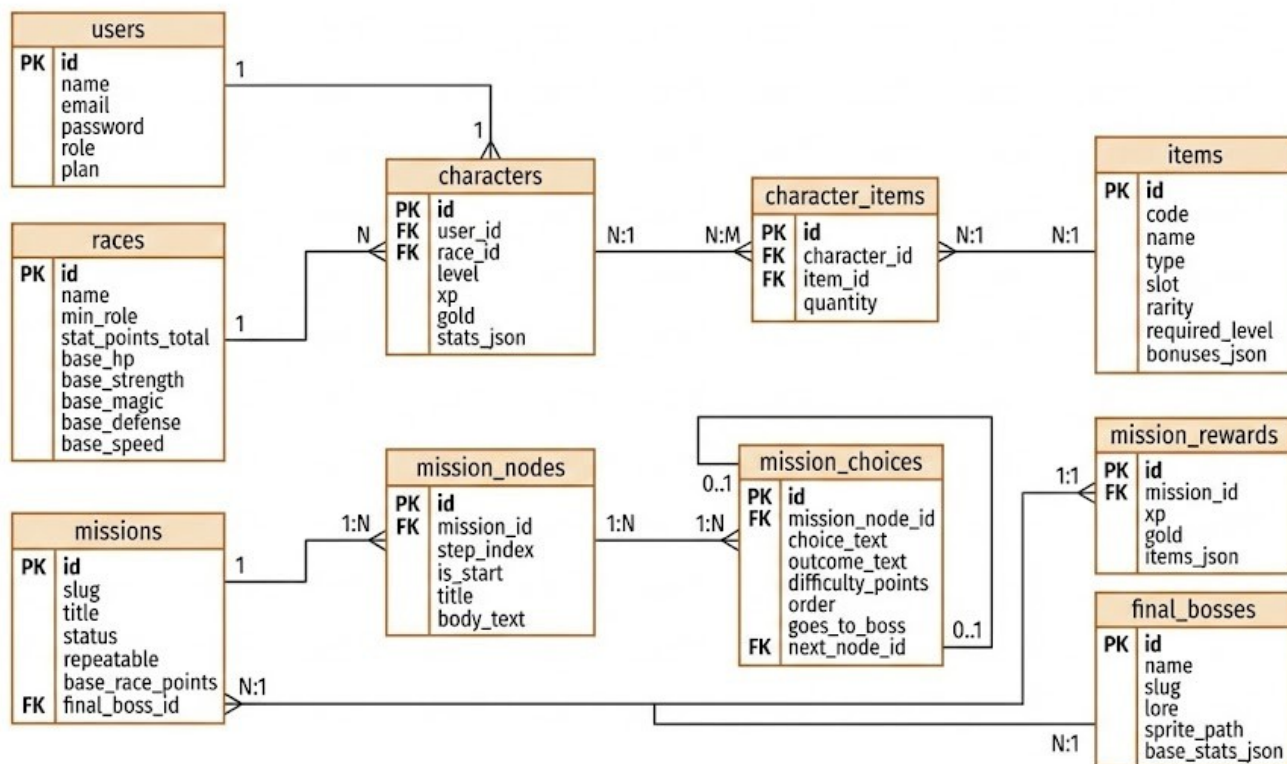
5.2 Explicación del ER

El ER no solo define “qué tablas existen”, sino por qué se han modelado así las relaciones para que el sistema sea coherente, jugable y mantenible. En un RPG, el dato no es decorativo: el modelo determina la estabilidad del juego, la integridad del progreso y la posibilidad de ampliar contenido sin romper el sistema.

Relaciones críticas justificadas:

- 1 usuario → 1 personaje
Esta relación garantiza que cada cuenta representa una única partida. Evita duplicidades de estado (dos progresos diferentes en el mismo usuario) y simplifica la lógica de juego: el inventario, el equipo, las estadísticas y el avance narrativo quedan siempre vinculados a una única identidad jugable. Además, permite aplicar restricciones claras y consistentes a la zona privada, ya que toda acción del jugador se apoya en un personaje único y verificable.
- Misiones como grafo (nodos/choices)
Las misiones no se modelan como una lista lineal de pasos, sino como una estructura ramificada compuesta por nodos y elecciones. Cada nodo representa un punto narrativo o de interacción, y cada elección actúa como enlace hacia el siguiente nodo. Este diseño permite implementar decisiones reales (ramificación), reutilizar nodos, controlar cierres y finales, y mantener coherencia en la navegación de la misión. A nivel conceptual, el ER refleja esa estructura para que el salto al modelo lógico (tablas + claves foráneas) sea directo y no dependa de soluciones “ad hoc”.
- Inventario y equipamiento
El inventario se modela como una relación N:M entre personaje e ítems, ya que un personaje puede tener múltiples ítems y el mismo tipo de ítem puede pertenecer a muchos personajes. Sobre ese inventario se apoya el equipamiento, que representa el conjunto de slots del personaje y qué ítems están equipados en cada uno. La separación conceptual inventario/equipamiento permite distinguir “poseer” de “usar”, y hace posible aplicar bonificaciones reales solo cuando el objeto está equipado, manteniendo el estado jugable consistente y evitando inconsistencias de estadísticas.

MODELO ENTIDAD-RELACIÓN (ER) — EL JURAMENTO DE VALTHERION



***Ilustración: Diagrama Entidad-Relación (ER)*



6. Modelo lógico (relacional)

6.1 Tablas del dominio (RPG)

El modelo lógico traduce el diseño conceptual (ER) a un esquema relacional listo para implementarse en un SGBD. En este proyecto, el objetivo es que la base de datos represente de forma fiable el estado jugable y la gestión de contenido, manteniendo consistencia en reglas clave: relación única usuario–personaje, misiones como grafo válido, y control real de inventario/equipamiento con efectos sobre estadísticas.

Tablas principales del dominio:

- **users (campos clave + rol/plan)**
Almacena la identidad del usuario y la información necesaria para control de acceso. Incluye campos clave de autenticación (email, password) y los atributos que determinan permisos y disponibilidad de contenido (rol y plan si aplica).
- **characters**
Representa el estado jugable persistente del usuario. Contiene la relación directa con el usuario (user_id), estadísticas, progreso, recursos y los valores necesarios para construir la ficha del personaje dentro del juego.
- **races**
Define las razas disponibles, sus atributos base y reglas asociadas (por ejemplo, reparto de puntos o restricciones por rol/plan). Esta tabla alimenta la creación y validación del personaje.
- **items (+ rarezas si aplica)**
Catálogo de objetos del juego. Incluye información de tipo (equipable/consumible), rareza, precio, bonificaciones y datos necesarios para tienda, inventario y equipamiento.
- **tabla pivote inventario (character_items)**
Relación N:M entre personajes e ítems. Permite reflejar posesión real del personaje sobre el catálogo de objetos (y, cuando aplica, cantidad, estado o metadatos del ítem asignado).
- **equipamiento (columns en characters o tabla aparte)**
Representa qué ítems están equipados en cada slot. Puede resolverse con columnas directas en characters por ranura, o con una tabla específica por slot. En ambos casos, separa “poseer” (inventario) de “usar” (equipo) para aplicar bonificaciones de forma coherente.
- **missions**
Define cada misión a nivel general: estado de publicación, información narrativa y metadatos necesarios para controlar acceso y progresión.

- **mission_nodes**
Nodos del grafo de misión. Cada nodo representa un paso narrativo o de interacción dentro de una misión y referencia a missions.
- **mission_choices**
Elecciones vinculadas a un nodo. Define las opciones disponibles y su enlace al siguiente nodo (next_node). Es la parte clave que hace que la misión sea ramificada.
- **mission_rewards**
Define recompensas asociadas a la misión o a su cierre (oro, experiencia, ítems u otros recursos), conectando finalización con progresión.
- **final_bosses**
Catálogo de jefes finales PvE. Incluye estadísticas y relación con misiones cuando el boss se usa como cierre.
- **(runs/historial)**
Si el proyecto registra intentos y resultados (misiones completadas, combates, PvP), se usa una tabla de historial para persistir esa información sin alterar el estado base del personaje.

6.2 Restricciones e integridad

El modelo lógico incorpora restricciones para asegurar que el estado jugable sea consistente y que el contenido administrado no permita datos inválidos. Esto es especialmente importante en un sistema con estructura de grafo (misiones) y relaciones de inventario/equipamiento.

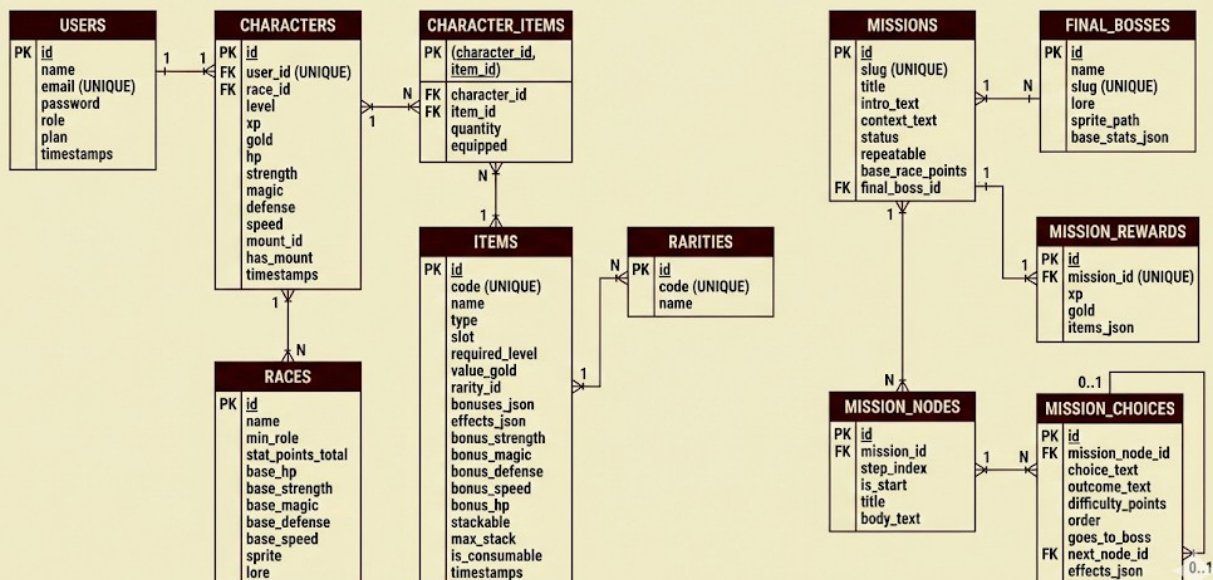
Restricciones principales:

- **Unicidades**
 - email UNIQUE en users para evitar duplicidades de cuenta.
 - slug UNIQUE en entidades que lo utilicen (misiones, bosses, ítems si aplica).
 - user_id UNIQUE en characters para garantizar 1:1 usuario–personaje.
- **Claves foráneas (FK) y reglas**
 - characters.user_id → users.id (obligatoria).
 - characters.race_id → races.id (obligatoria).
 - character_items.character_id → characters.id y character_items.item_id → items.id (obligatorias).
 - mission_nodes.mission_id → missions.id (obligatoria).



- mission_choices.mission_node_id → mission_nodes.id (obligatoria).
- mission_choices.next_node_id → mission_nodes.id (nullable) para permitir finales o transiciones fuera del grafo cuando corresponda.
- mission_rewards vinculado a missions (y a lo que aplique según diseño) con FK coherentes.
- Índices recomendados
 - Índices en columnas de relación frecuente: user_id (characters), mission_id (mission_nodes), mission_node_id (mission_choices), character_id e item_id (character_items).
 - Índices en campos usados para listados/filtrado: estado de misión (draft/published), rareza, tipo de ítem, etc.
 - Índices compuestos cuando corresponda (por ejemplo, character_id + item_id en la pivote para acelerar consultas y evitar duplicados lógicos).

Ilustración 5 — Modelo Lógico (Relacional) — El Juramento de Valtherion



****Ilustración Modelo Lógico del sistema (Relacional)**



7. Manual de instalación

7.1 Requisitos

Para ejecutar *El Juramento de Valtherion* en un entorno local de evaluación (Windows), se necesita disponer de un stack compatible con Laravel 12 y con SQLite como motor de persistencia en fichero. El proyecto se ha preparado para funcionar sin servicios externos (no hay MySQL), por lo que el requisito clave es que PHP tenga activas las extensiones de SQLite.

Requisitos de software:

- XAMPP (o, como mínimo, PHP CLI operativo con un servidor local)
- PHP compatible con Laravel 12 (entorno usado: PHP 8.2+)
- Composer (dependencias backend)
- Node.js + NPM (dependencias frontend y compilación con Vite)
- Git (solo necesario si se instala por clonado)
- Navegador moderno (Chrome/Edge/Firefox) para pruebas

Extensiones de PHP necesarias (especialmente importantes para SQLite):

- pdo_sqlite
- sqlite3
- openssl, mbstring, fileinfo, tokenizer, ctype, json

7.2 Instalación por clonado

Este método deja el proyecto exactamente como está en el repositorio y asegura que la estructura sea la correcta desde el inicio.

Pasos:

- Abrir PowerShell y entrar en la ruta de trabajo:

```
cd C:\xampp\htdocs
```

- Clonar el repositorio y entrar en la carpeta:

```
git clone https://github.com/kampexiii/proyectoElJuramentoDeValtherion elJuramentoDeValtherion
```

```
cd elJuramentoDeValtherion
```

- Instalar dependencias del backend:

```
composer install
```




- Instalar dependencias del frontend:

`npm install`

7.3 Instalación por ZIP

Este método es el más seguro y rapido para principiantes: se descomprime el proyecto completo y se levanta en local.

Puntos críticos para evitar errores de ruta:

- La carpeta raíz del proyecto debe contener directamente:
 - artisan
 - composer.json
 - app/, routes/, database/, resources/

Ruta recomendada:

- C:\xampp\htdocs\elJuramentoDeValtherion

Cuando al descomprimir queda una carpeta duplicada (por ejemplo ... \elJuramentoDeValtherion\elJuramentoDeValtherion\artisan), se corrige moviendo el contenido una carpeta hacia arriba hasta que artisan quede en el nivel correcto.

Instalación de dependencias cuando no se entregan incluidas:

`composer install`

`npm install`

****En este caso se da todo incluido y listo.**

7.4 Configuración SQLite

El proyecto usa SQLite como fichero portable, con el archivo renombrado a:

- database/db_proyecto_sevillano.sqlite

Configuración de entorno (.env):

- DB_CONNECTION=sqlite
- DB_DATABASE=database/db_proyecto_sevillano.sqlite

Preparación del .env (si no existe):

`copy .env.example .env`

`php artisan key:generate`



Creación del fichero SQLite (si no está creado aún en database/):

```
New-Item -ItemType File .\database\db_proyecto_sevillano.sqlite -Force
```

7.5 Migraciones y seeders

La base de datos se construye desde migraciones y se carga contenido inicial con seeders. Para evaluación, el flujo recomendado es regenerar la BD limpia con datos completos, garantizando consistencia.

Comando recomendado (reinicio completo + seeders):

```
php artisan migrate:fresh --seed
```

Limpieza de cachés para evitar rutas/vistas/config desactualizadas:

```
php artisan optimize:clear
```

7.6 Assets

El proyecto compila CSS/JS con Vite.

Modo evaluación (sin dejar procesos en segundo plano):

```
npm run build
```

Modo desarrollo (recarga en caliente mientras se trabaja):

```
npm run dev
```

7.7 Arranque del proyecto

Ejecución local con Artisan:

```
php artisan serve
```

La aplicación queda accesible en:

- <http://127.0.0.1:8000>

7.8 Reverb / Chat

El chat usa Broadcasting con Reverb. Para que el módulo funcione, Reverb debe estar arrancado en una terminal adicional mientras la aplicación está en ejecución.

Arranque de Reverb:

```
php artisan reverb:start
```

Notas de ejecución en Windows:

- Permitir el acceso en el firewall cuando Windows lo solicita (red privada / localhost).
- Mantener libres los puertos configurados para el servicio.

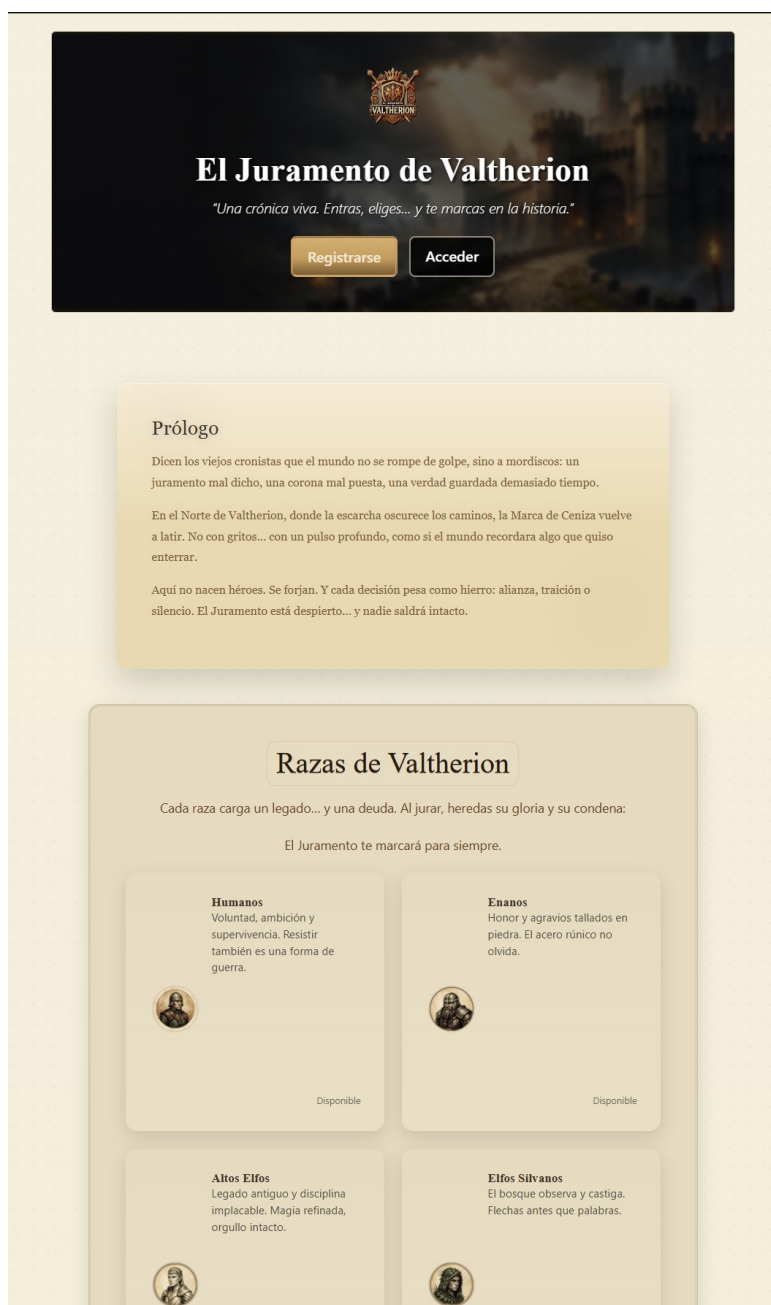


7.9 Usuarios de prueba

Tras ejecutar seeders, queda disponible un usuario administrador para acceder al panel y gestionar contenido:

- Admin:
 - Email: pablosevillanoa90@gmail.com
 - Password: 12345678
 - Role: admin
 - Plan: premium

Además, cualquier jugador puede registrarse desde la aplicación usando el formulario de registro.



8. Manual de usuario

Regla aplicada en todas las pantallas: en cada apartado se describe qué hace la pantalla, cómo se usa (pasos), qué validaciones/mensajes aparecen, qué rol la visualiza y el marcador de captura correspondiente.

8.1 Home pública / Lore (visitante)

Qué hace

La Home pública actúa como puerta de entrada al proyecto. Presenta el universo (lore), contextualiza la aplicación y ofrece accesos directos a las acciones iniciales: registrarse o iniciar sesión. Desde esta zona también se puede navegar a las páginas legales para consultar privacidad, cookies y términos.

Pasos de uso

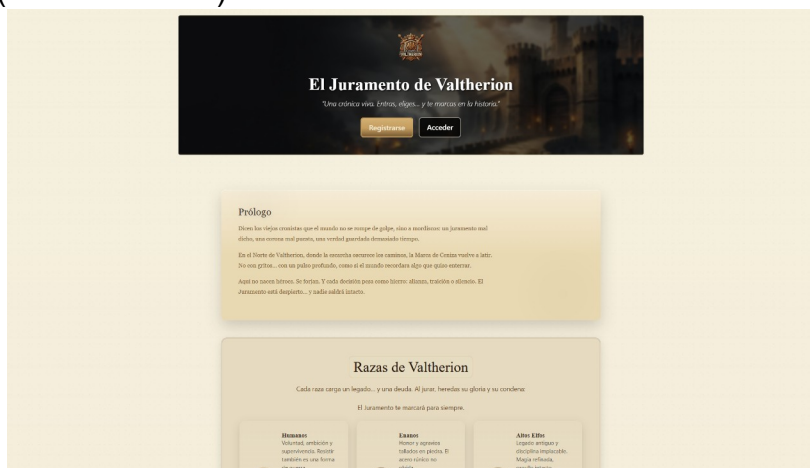
- Acceder a la URL principal del proyecto en navegador.
- Leer el contenido introductorio (lore/presentación).
- Usar los accesos visibles para:
 - ir a Registro, o
 - ir a Login.

Validaciones / mensajes

- No requiere validación porque no hay acciones críticas obligatorias en esta pantalla.
- Enlaces disponibles y navegación directa sin restricciones.

Rol que la ve

- Visitante (no autenticado).



****Ilustración: Home pública / Lore**



8.2 Registro

Qué hace

Permite crear una cuenta de jugador para acceder a la zona privada. El registro es el punto de entrada a toda la experiencia jugable, ya que sin cuenta no se puede crear personaje ni acceder a los módulos internos.

Pasos de uso

- Acceder a la pantalla de Registro desde la Home pública.
- Introducir los datos requeridos (nombre, email y contraseña).
- Enviar el formulario para crear la cuenta.
- Tras el alta correcta, continuar al inicio de sesión o al flujo definido de acceso.

Validaciones / mensajes

- Campos obligatorios: si faltan, se muestran mensajes de validación.
- Formato de email: se valida que sea correcto.
- Email único: si ya existe una cuenta con ese email, se muestra el aviso correspondiente.
- Contraseña: se valida longitud y requisitos definidos en el sistema.

Rol que la ve

- Visitante (no autenticado).

The screenshot shows a registration form for 'Valtherion' on a dark background. The form is centered and contains the following fields and elements:

- Nombre:** A text input field with the placeholder 'Ej: Valtherion'.
- Correo Electrónico:** A text input field with the placeholder 'Ej: usuario@dominio.com'.
- Contraseña:** A text input field with the placeholder 'Minimo 8 caracteres'.
- Confirmar Contraseña:** A text input field with the placeholder 'Repite la contraseña'.
- ☐ Acepto el uso de cookies. [Ver cookies](#)
- ☐ Acepto los terminos y condiciones. [Ver terminos](#)
- Below the checkboxes, it says '¿Ya estás registrado?'.
- A blue button labeled 'REGISTRARSE' is located at the bottom right of the form.

***Ilustración: Registro*

8.3 Login

Qué hace

Permite acceder a la zona privada del proyecto mediante autenticación. Una vez iniciada la sesión, el sistema habilita el acceso a la experiencia de juego. Si el usuario tiene rol administrador, además se habilita el acceso a la zona admin.

Pasos de uso

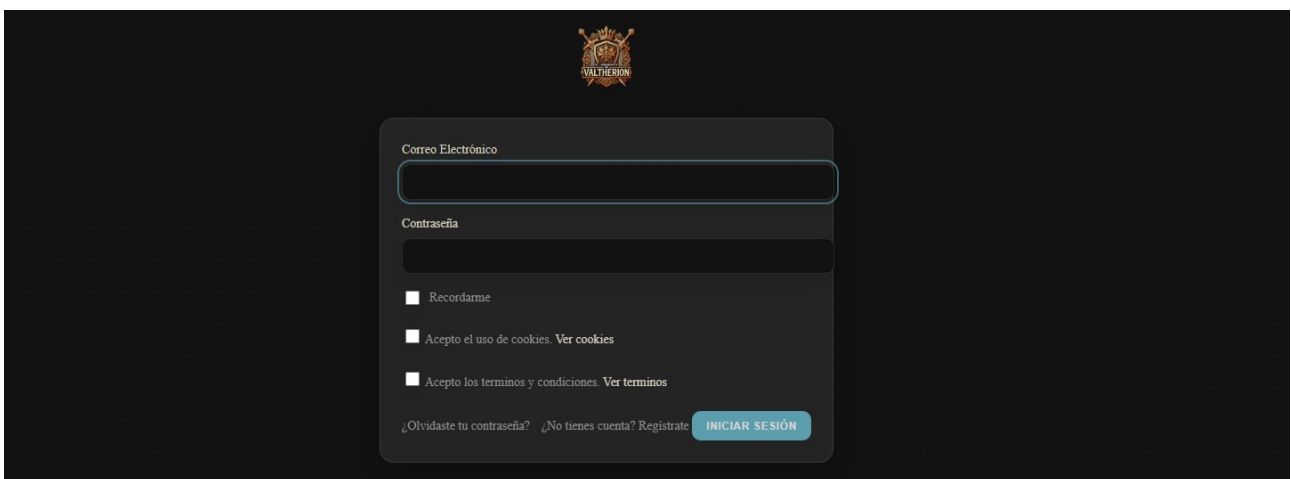
- Acceder a la pantalla de Login desde la Home pública.
- Introducir email y contraseña.
- Enviar el formulario para iniciar sesión.
- Tras autenticación correcta:
 - acceder a la zona privada,
 - si es primer acceso, continuar a creación de personaje,
 - si ya existe personaje, ir al Home del juego.

Validaciones / mensajes

- Credenciales inválidas: se muestra mensaje de error sin revelar detalles sensibles.
- Campos vacíos: validación obligatoria.
- Sesión iniciada correctamente: redirección a la zona privada.

Rol que la ve

- Visitante (no autenticado).



***Ilustración: Login*



8.4 Home del juego (jugador)

Qué hace

Es el panel principal de navegación una vez dentro del juego. Centraliza accesos a los módulos: misiones, combate, inventario, equipamiento, tienda, PvP y chat. También muestra información base del personaje para que el jugador tenga visión rápida de su estado.

Pasos de uso

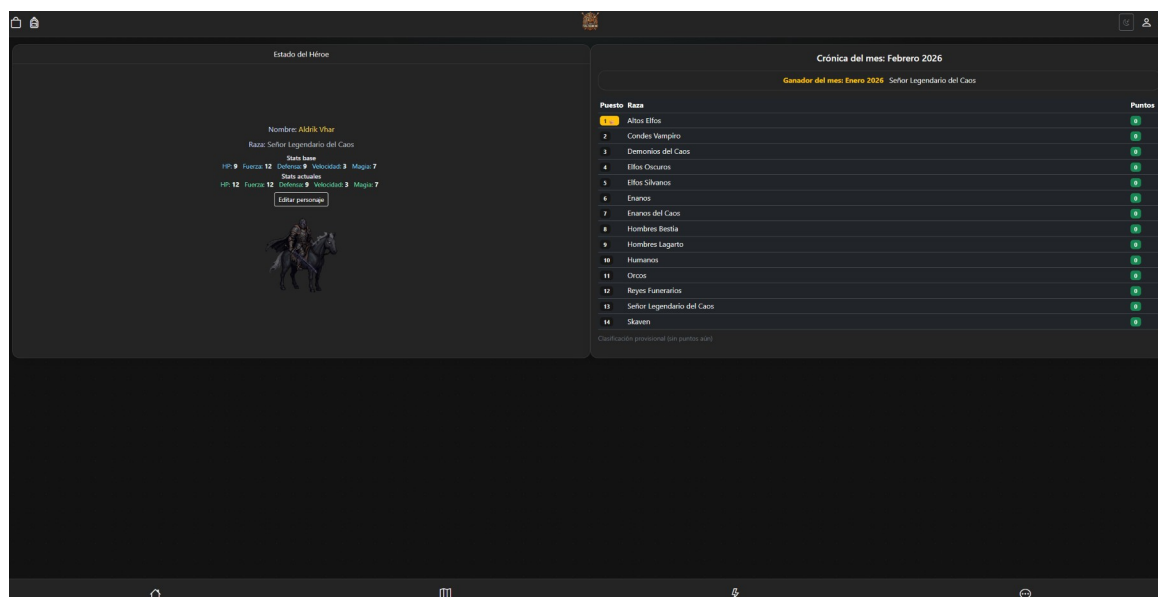
- Iniciar sesión.
- Acceder al Home del juego automáticamente tras el flujo interno.
- Desde el menú o accesos del panel, entrar al módulo deseado.
- Volver al Home para cambiar de módulo cuando sea necesario.

Validaciones / mensajes

- Acceso restringido: si no hay sesión iniciada, se bloquea la ruta y se redirige a Login.
- Control de personaje: si no existe personaje, el sistema conduce a creación de personaje antes de permitir el acceso a módulos.

Rol que la ve

- Jugador autenticado (usuario estándar o premium) y administrador cuando accede como jugador.



***Ilustración: Home del juego (panel principal)*

8.5 Creación de personaje + selección de raza

Qué hace

En el primer acceso a la zona privada, se fuerza la creación del personaje. Este personaje representa el estado jugable completo: estadísticas, progresión, inventario, recursos y relación con la raza elegida. Durante este paso se selecciona la raza y se realiza el reparto inicial de puntos bajo reglas de acceso y coherencia.

Pasos de uso

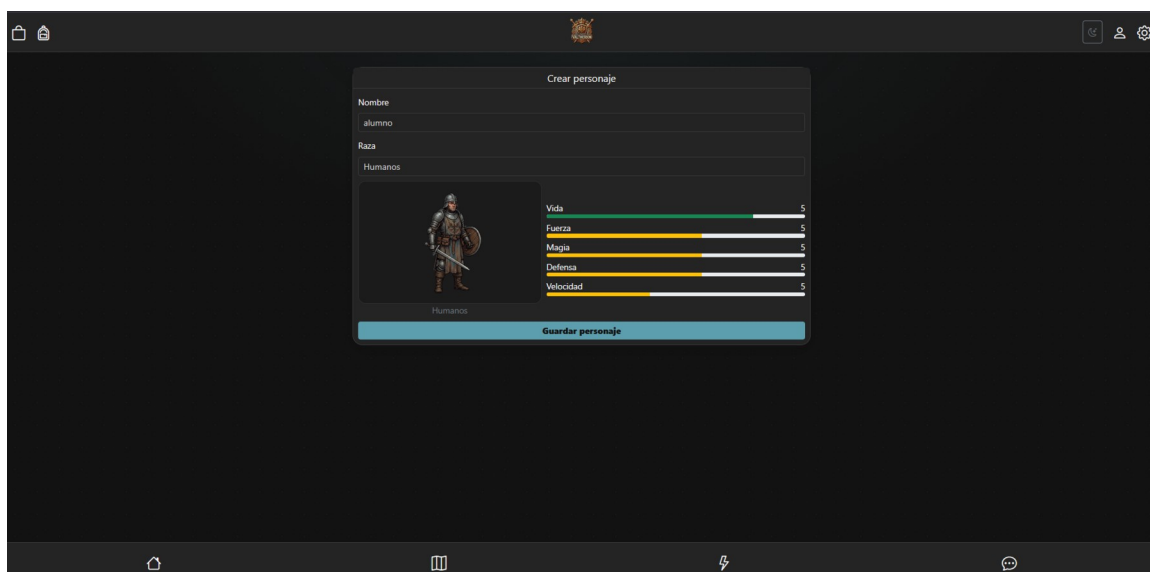
- Entrar a la zona privada por primera vez tras Login.
- Abrir el formulario de creación de personaje (flujo forzado).
- Seleccionar una raza disponible según rol/plan.
- Repartir los puntos iniciales en las estadísticas indicadas.
- Confirmar la creación para guardar el personaje y acceder al Home del juego.

Validaciones / mensajes

- Reparto inválido: se impide guardar si no se cumple el total de puntos o límites definidos.
- Restricción de razas: solo se muestran/permiten razas válidas para ese usuario.
- Relación 1:1: el sistema evita que un usuario cree más de un personaje.

Rol que la ve

- Jugador autenticado que aún no tiene personaje.



***Ilustración: Creación de personaje y selección de raza*



8.6 Misiones (listado)

Qué hace

Muestra el catálogo de misiones disponibles para el jugador. Desde aquí se selecciona una misión para comenzar el recorrido narrativo ramificado.

Pasos de uso

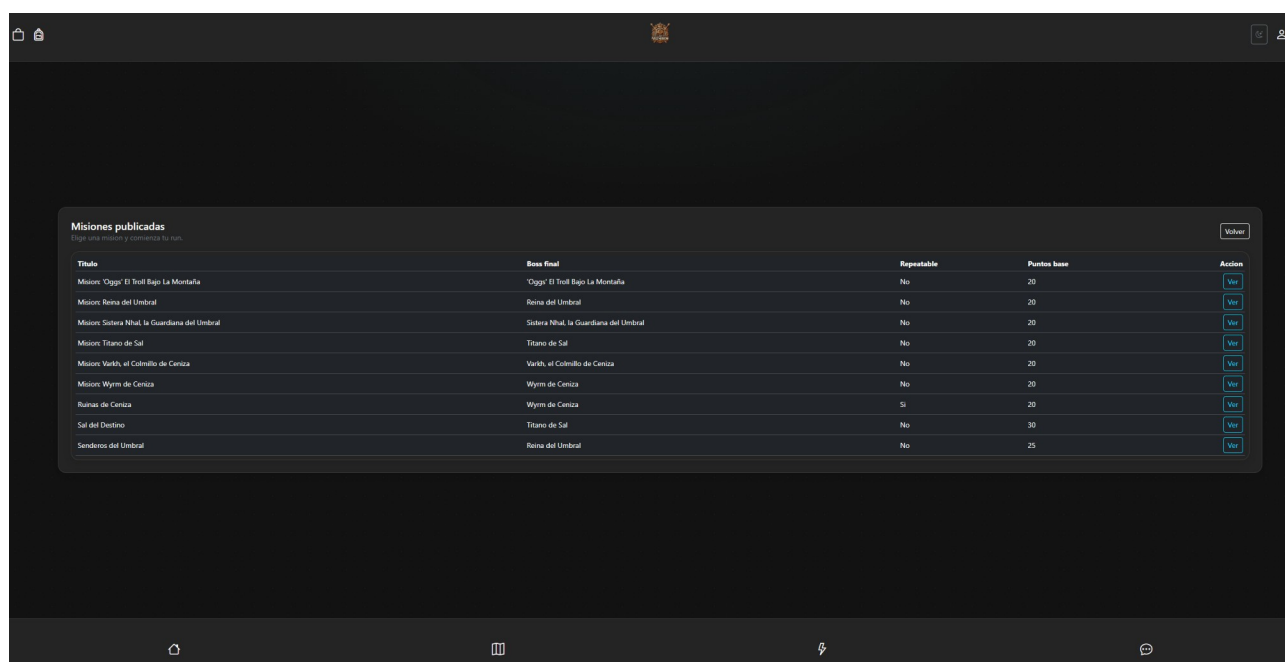
- Entrar en el módulo Misiones desde el Home del juego.
- Consultar las misiones disponibles.
- Seleccionar una misión para iniciar su ejecución.

Validaciones / mensajes

- Solo misiones publicadas: el listado respeta el estado de publicación para evitar contenido incompleto.
- Acceso restringido a usuarios autenticados.

Rol que la ve

- Jugador autenticado.



Título	Base final	Repeatable	Puntos base	Acción
Misión: 'Oggir' El Trío Bajo La Montaña	'Oggir' El Trío Bajo La Montaña	No	20	Ver
Misión: Reina del Umbral	Reina del Umbral	No	20	Ver
Misión: Sistema Nhal, la Guardiara del Umbral	Sistema Nhal, la Guardiara del Umbral	No	20	Ver
Misión: Titano de Sal	Titano de Sal	No	20	Ver
Misión: Varkh, el Colmillo de Ceniza	Varkh, el Colmillo de Ceniza	No	20	Ver
Misión: Wyrm de Ceniza	Wyrm de Ceniza	No	20	Ver
Rutras de Ceniza	Wyrm de Ceniza	Si	20	Ver
Sal del Destino	Titano de Sal	No	30	Ver
Senderos del Umbral	Reina del Umbral	No	25	Ver

***Ilustración: Listado de misiones*

8.7 Misión en ejecución (nodo + elecciones)

Qué hace

Permite recorrer la misión como un grafo narrativo: el jugador lee el texto del nodo actual y elige entre varias opciones. Cada elección aplica su dificultad/consecuencia y conduce a un siguiente nodo o al cierre de misión.

Pasos de uso

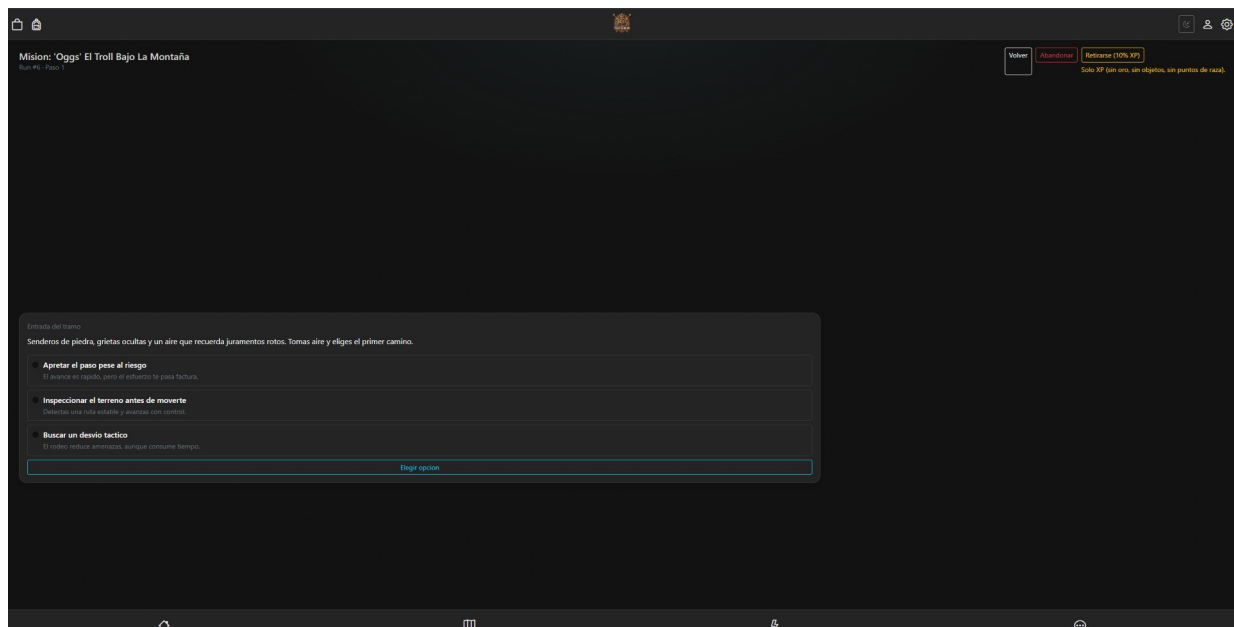
- Iniciar una misión desde el listado.
- Leer el texto del nodo mostrado.
- Elegir una opción disponible.
- Repetir el proceso avanzando nodo a nodo hasta alcanzar el final definido.

Validaciones / mensajes

- Elección obligatoria: no se avanza sin seleccionar una opción.
- Coherencia del grafo: las elecciones conducen a nodos válidos o al final previsto.
- Mensajes de resultado: se muestra el texto/consecuencia asociado a la elección.

Rol que la ve

- Jugador autenticado.



***Ilustración: Misión en ejecución (nodo y elecciones)*

8.8 Combate PvE (boss)

Qué hace

Representa el enfrentamiento final contra el boss asociado a la misión. Es un combate orientado a turnos, donde el jugador ejecuta acciones y el sistema resuelve el intercambio hasta determinar victoria o derrota. Al finalizar, se aplican recompensas y cambios de estado.

Pasos de uso

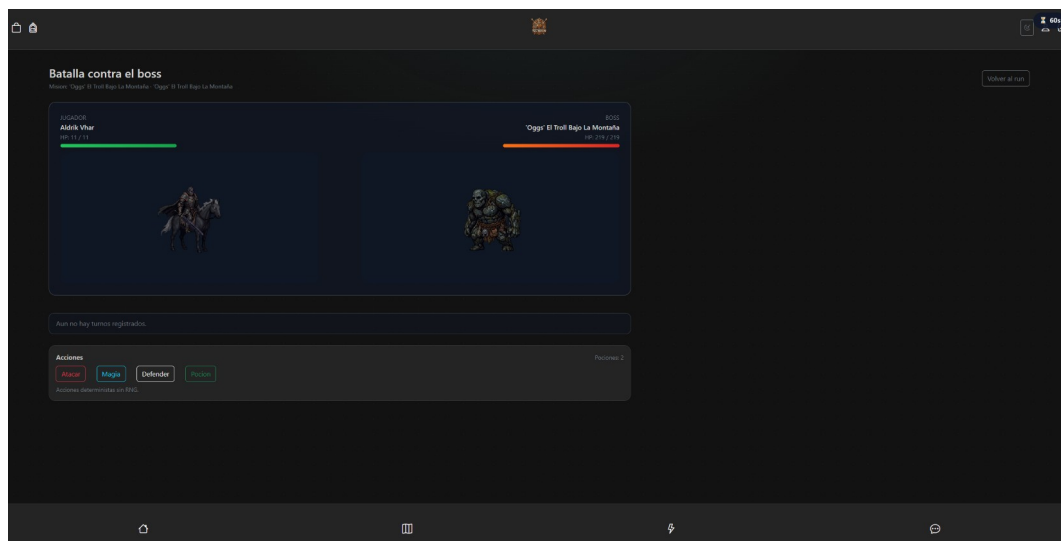
- Llegar al tramo final de una misión (transición a boss).
- Entrar en la pantalla de combate.
- Seleccionar acciones disponibles en cada turno.
- Continuar hasta finalizar el combate.
- Recibir resultado y volver a la ruta indicada por el sistema.

Validaciones / mensajes

- Acceso condicionado: el combate se activa desde el flujo de misión.
- Mensajes de resultado: victoria/derrota y aplicación de recompensas o consecuencias.
- Control de estado: no se permite repetir turnos inválidos.

Rol que la ve

- Jugador autenticado.



****Ilustración: Combate PvE contra boss**

8.9 Inventario

Qué hace

Muestra los ítems que posee el personaje, incluyendo equipables y consumibles. Desde aquí se consulta el detalle de cada objeto y, cuando corresponde, se realizan acciones como usar consumibles o gestionar equipamiento.

Pasos de uso

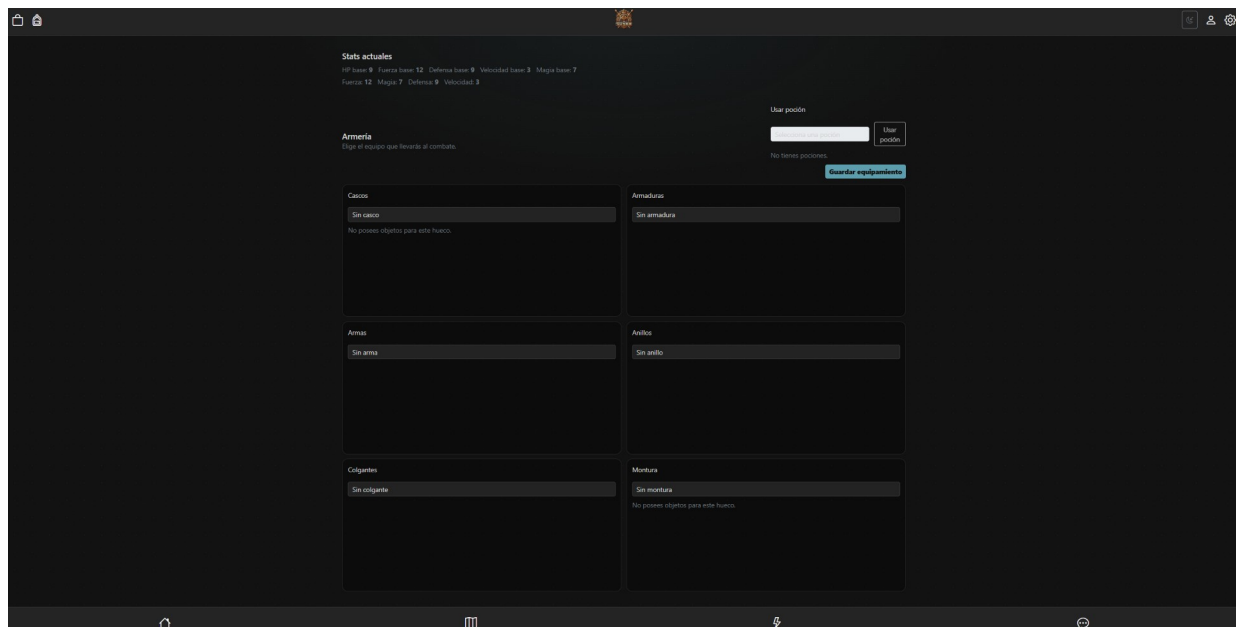
- Acceder a Inventario desde el Home del juego.
- Revisar el listado de ítems disponibles.
- Consultar detalle del ítem (rareza, bonus, descripción).
- Ejecutar acciones disponibles según el tipo de ítem (usar/equipar si corresponde).

Validaciones / mensajes

- Uso de consumibles: si no se cumplen condiciones, se muestra aviso.
- Inventario vacío: se refleja de forma clara.
- Acceso restringido a usuarios autenticados.

Rol que la ve

- Jugador autenticado.



***Ilustración: Inventario*

8.10 Equipamiento

Qué hace

Permite equipar objetos por slots (arma, armadura, casco, anillo, amuleto, montura). Al equipar, se aplican bonificaciones reales sobre las estadísticas del personaje y el sistema mantiene el estado consistente (un ítem por ranura).

Pasos de uso

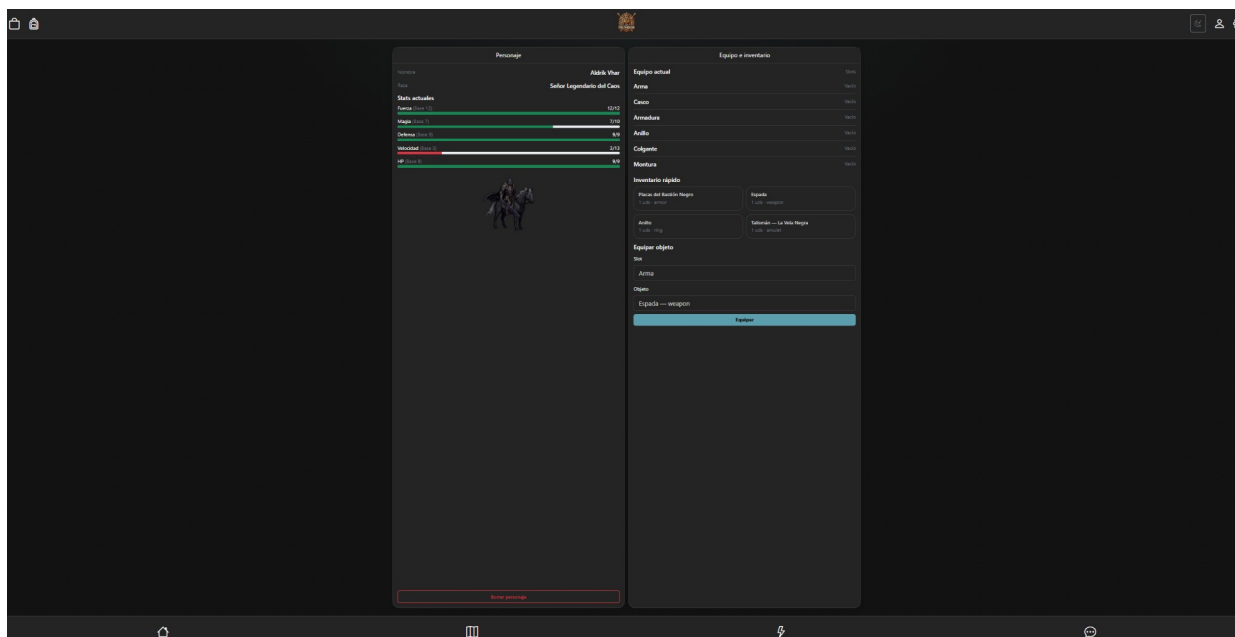
- Acceder a Equipamiento desde el Home del juego.
- Revisar los slots disponibles.
- Seleccionar un ítem compatible para equipar en el slot correspondiente.
- Confirmar el cambio y comprobar estadísticas actualizadas.
- Retirar o sustituir equipamiento cuando sea necesario.

Validaciones / mensajes

- Compatibilidad de slot: no se permite equipar un ítem en una ranura incorrecta.
- Actualización de stats: se refleja el efecto del equipo sobre el personaje.
- Acceso restringido a usuarios autenticados.

Rol que la ve

- Jugador autenticado.



***Ilustración: Equipamiento por slots*

8.11 Tienda

Qué hace

Permite comprar ítems del catálogo usando la moneda interna del personaje. Los objetos adquiridos pasan al inventario y pueden influir en el progreso (equipables) o apoyar en combate/misiones (consumibles).

Pasos de uso

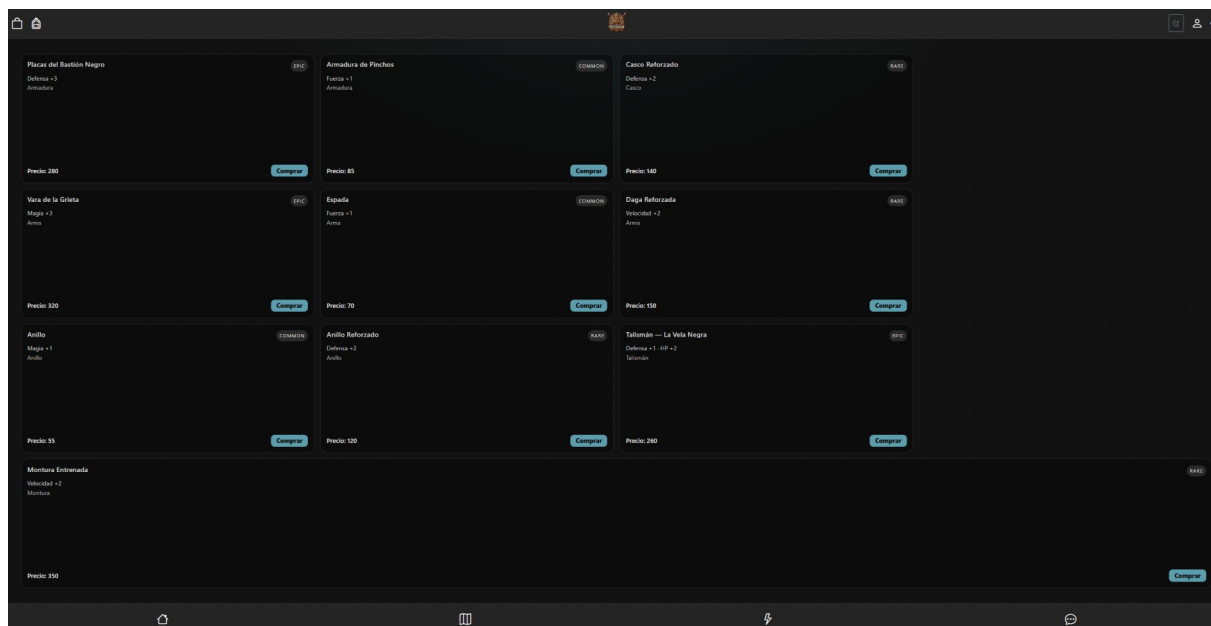
- Acceder a Tienda desde el Home del juego.
- Revisar catálogo (precio, rareza, bonus).
- Seleccionar un ítem para comprar.
- Confirmar compra.
- Ver el ítem reflejado en inventario y el oro actualizado.

Validaciones / mensajes

- Oro insuficiente: se bloquea la compra y se muestra aviso.
- Compra correcta: confirmación y actualización de recursos.
- Acceso restringido a usuarios autenticados.

Rol que la ve

- Jugador autenticado.



***Ilustración: Tienda*



8.12 PvP

Qué hace

Ofrece el módulo de enfrentamiento jugador contra jugador como extensión competitiva del sistema. Permite acceder al flujo PvP definido y resolver un combate entre jugadores con sus estadísticas y equipamiento.

Pasos de uso

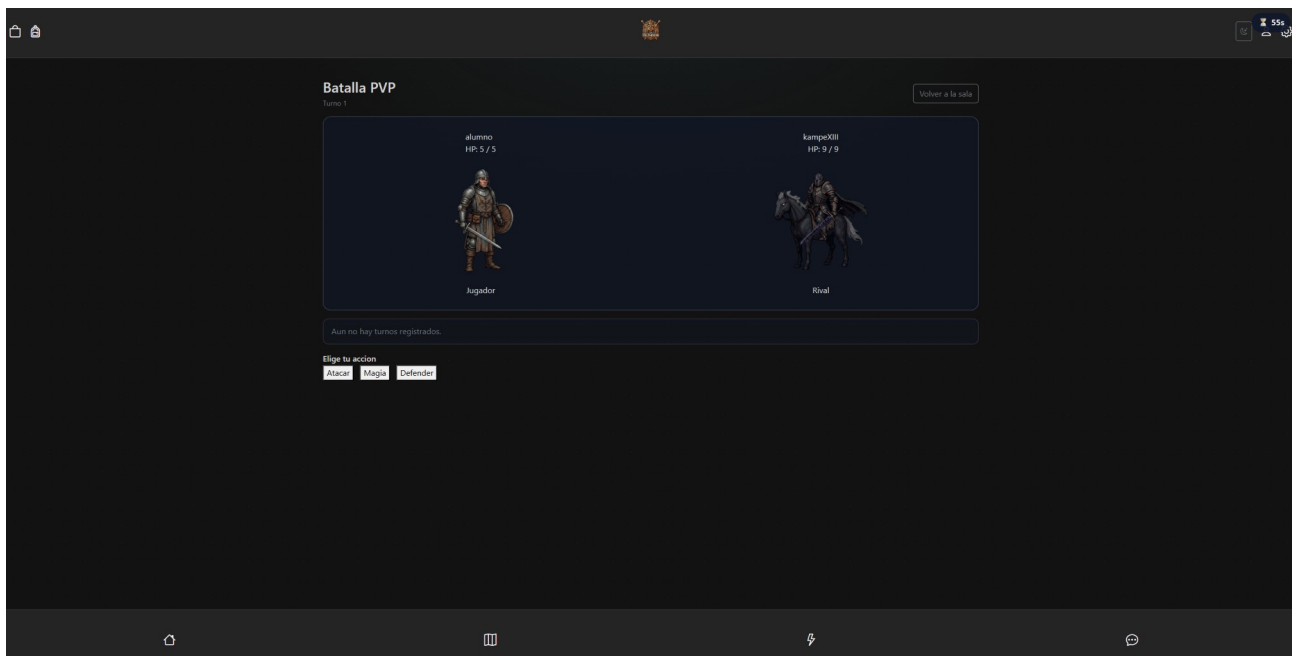
- Acceder a PvP desde el Home del juego.
- Entrar en el flujo disponible (por ejemplo lobby/sala).
- Seleccionar rival o acceder a una sala.
- Ejecutar el combate PvP según el sistema.
- Finalizar y volver al panel del juego.

Validaciones / mensajes

- Acceso restringido a usuarios autenticados.
- Mensajes de estado: espera, emparejamiento, resultado del combate.

Rol que la ve

- Jugador autenticado.



***Ilustración: PvP*



8.13 Chat

Qué hace

Permite comunicación interna dentro del juego. El chat funciona mediante tiempo real cuando el servicio de Reverb está arrancado, mostrando mensajes en vivo y permitiendo enviar nuevos mensajes al canal definido.

Pasos de uso

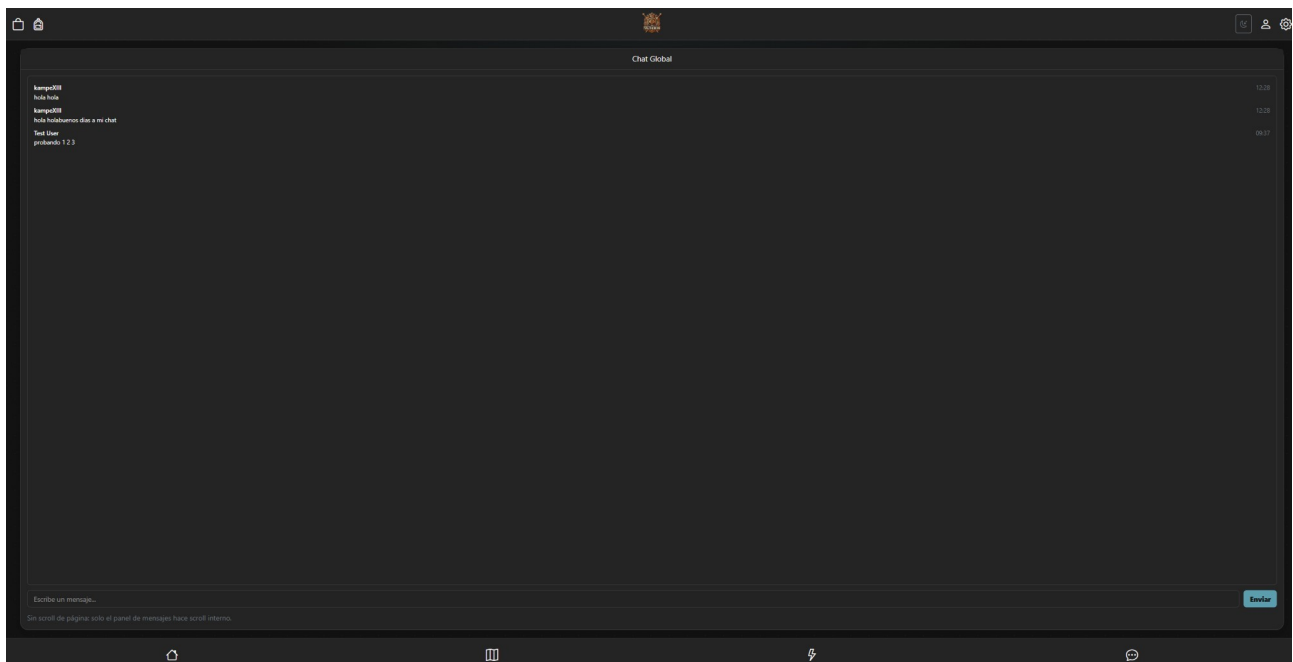
- Acceder a Chat desde el Home del juego.
- Visualizar mensajes existentes.
- Escribir un mensaje y enviarlo.
- Ver el mensaje reflejado en el canal.

Validaciones / mensajes

- Envío vacío: se valida que el mensaje tenga contenido.
- Servicio tiempo real: Para probar el chat en tiempo real es necesario arrancar Reverb; sin el servicio activo no se reciben mensajes en vivo.
- Acceso restringido a usuarios autenticados.

Rol que la ve

- Jugador autenticado.



***Ilustración: Chat interno*



8.14 Páginas legales

Qué hace

Permite consultar la información legal del proyecto: privacidad, cookies y términos. Son accesibles desde la zona pública y sirven como cierre documental y de transparencia del sistema.

Pasos de uso

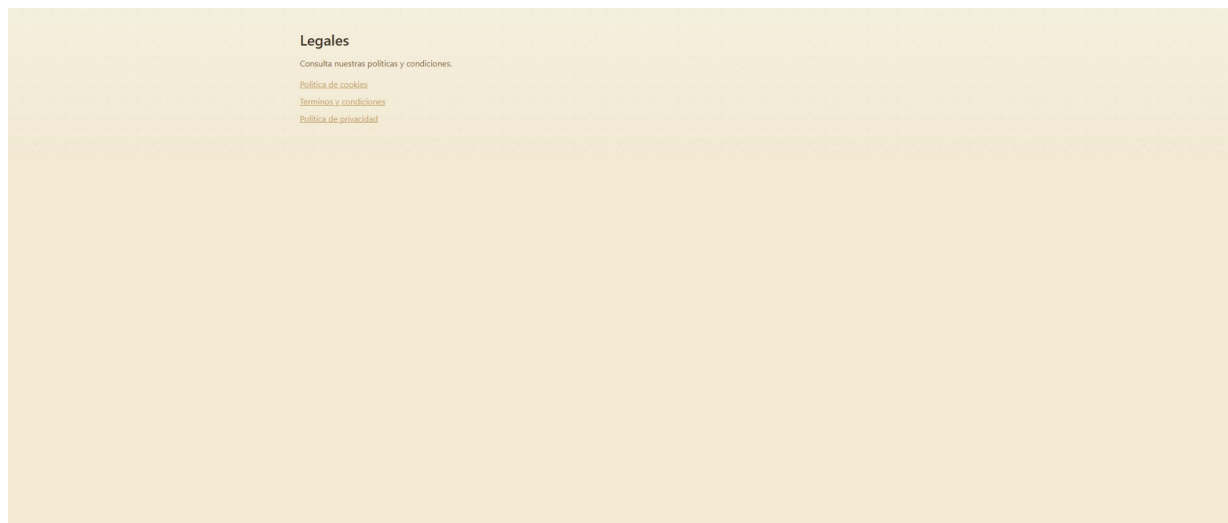
- Acceder desde enlaces de navegación (normalmente en la Home pública).
- Entrar en Privacidad, Cookies o Términos.
- Leer el contenido informativo.

Validaciones / mensajes

- No requieren validación.
- Acceso libre desde zona pública.

Rol que la ve

- Visitante y usuarios autenticados (cuando el enlace esté disponible desde la navegación).



****Ilustración: Páginas legales**

9. Manual de administrador

Regla aplicada en todas las pantallas: en cada apartado se describe qué hace la pantalla, cómo se usa (pasos), qué validaciones/mensajes aparecen, qué rol la visualiza y el marcador de captura correspondiente.

9.1 Acceso al panel admin

Qué hace

Permite entrar al panel de administración del proyecto, separado de la zona jugable. Desde aquí se gestiona el contenido del juego y los usuarios, manteniendo el sistema ampliable sin tocar directamente la base de datos.

Pasos de uso

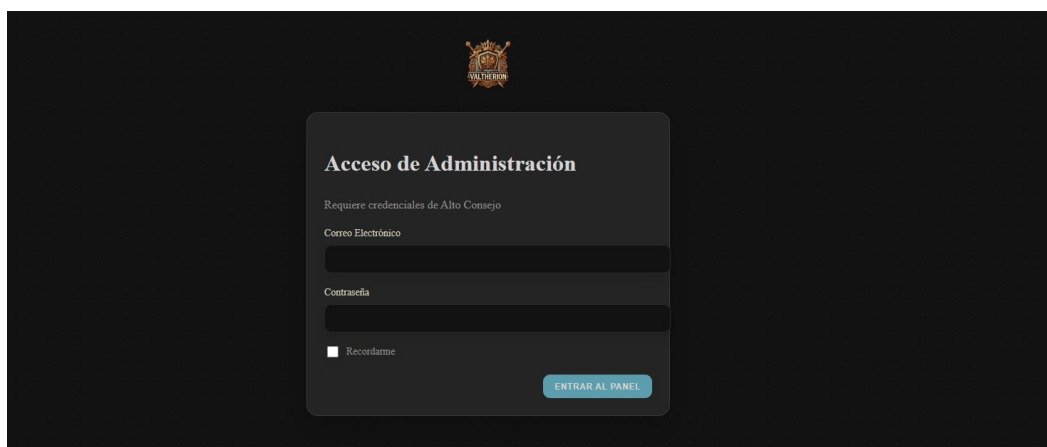
- Iniciar sesión con una cuenta con rol administrador.
- Acceder a la ruta del panel (/admin) desde navegación disponible o URL directa.
- Verificar que se muestra el dashboard/menú de administración con los módulos disponibles.

Validaciones / mensajes

- Control de acceso por rol: si el usuario no es administrador, el sistema bloquea el acceso y redirige a la zona permitida.
- Acceso sin sesión: redirección a login.
- Mensajes de permisos insuficientes cuando corresponda.

Rol que la ve

- Administrador.



***Ilustración: Acceso al panel admin*



9.2 Gestión de bosses (CRUD)

Qué hace

Permite crear, listar, editar y eliminar bosses finales del juego. Los bosses están vinculados al cierre PvE de las misiones y definen atributos de combate, recompensas y comportamiento general dentro del módulo PvE.

Pasos de uso

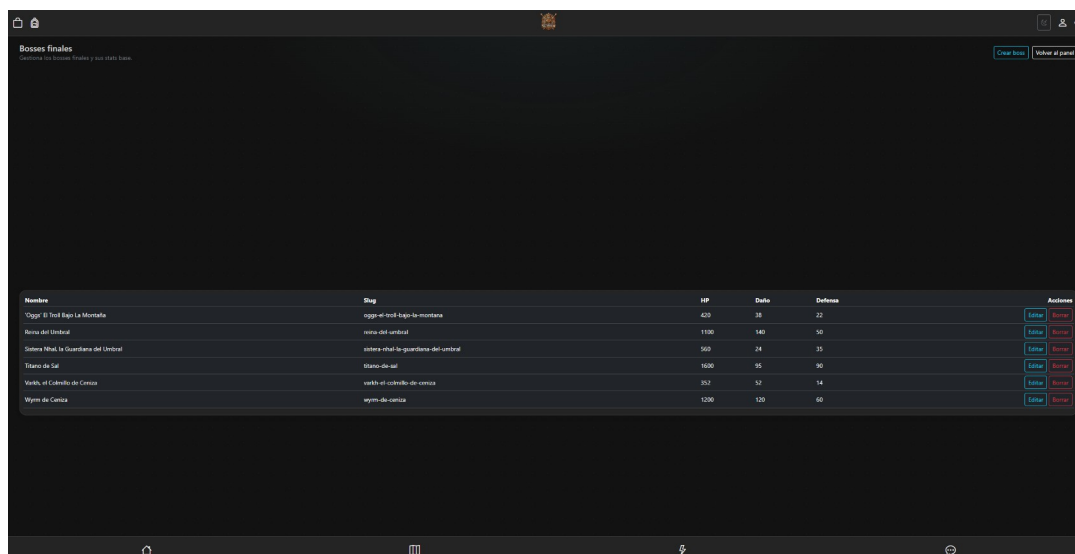
- Entrar al módulo “Bosses” desde el panel admin.
- Crear un boss nuevo completando los campos requeridos.
- Editar un boss existente para ajustar estadísticas, recompensas o configuración.
- Eliminar un boss cuando sea necesario, manteniendo coherencia con el contenido publicado.

Validaciones / mensajes

- Campos obligatorios: se valida que la información crítica no quede vacía.
- Coherencia de datos: valores numéricos dentro de límites definidos (vida, daño, recompensas, etc.).
- Eliminación controlada: si el boss está vinculado a contenido que dependa de él, se gestionan restricciones o avisos según reglas del sistema.

Rol que la ve

- Administrador.



The screenshot shows the 'Bosses finales' (Final Bosses) management interface. It features a table with columns for Name, Slug, HP, Damage, Defense, and Actions. The table lists six bosses with their respective attributes and edit/delete options.

Nombre	Slug	HP	Daño	Defensa	Acciones
Uggir el Troll Bajo la Montaña	uggir-el-troll-bajo-la-montana	420	35	22	Editar Eliminar
Brama del Umbral	brama-del-umbral	1100	140	50	Editar Eliminar
Centina Nihil, la Guardiadora del Umbral	centina-nihil-la-guardiadora-del-umbral	560	24	35	Editar Eliminar
Titano de Sal	titano-de-sal	1600	95	90	Editar Eliminar
Vandir, el Caballero de Coraza	vandir-el-caballero-de-coraza	392	52	14	Editar Eliminar
Wyrn de Caza	wyrn-de-caza	1200	120	60	Editar Eliminar

***Ilustración: CRUD bosses*



9.3 Gestión de misiones (CRUD)

Qué hace

Permite administrar las misiones principales del juego: creación, edición, configuración y control de estados (por ejemplo borrador/publicada). Las misiones actúan como contenedor de nodos y elecciones y definen la estructura jugable del recorrido.

Pasos de uso

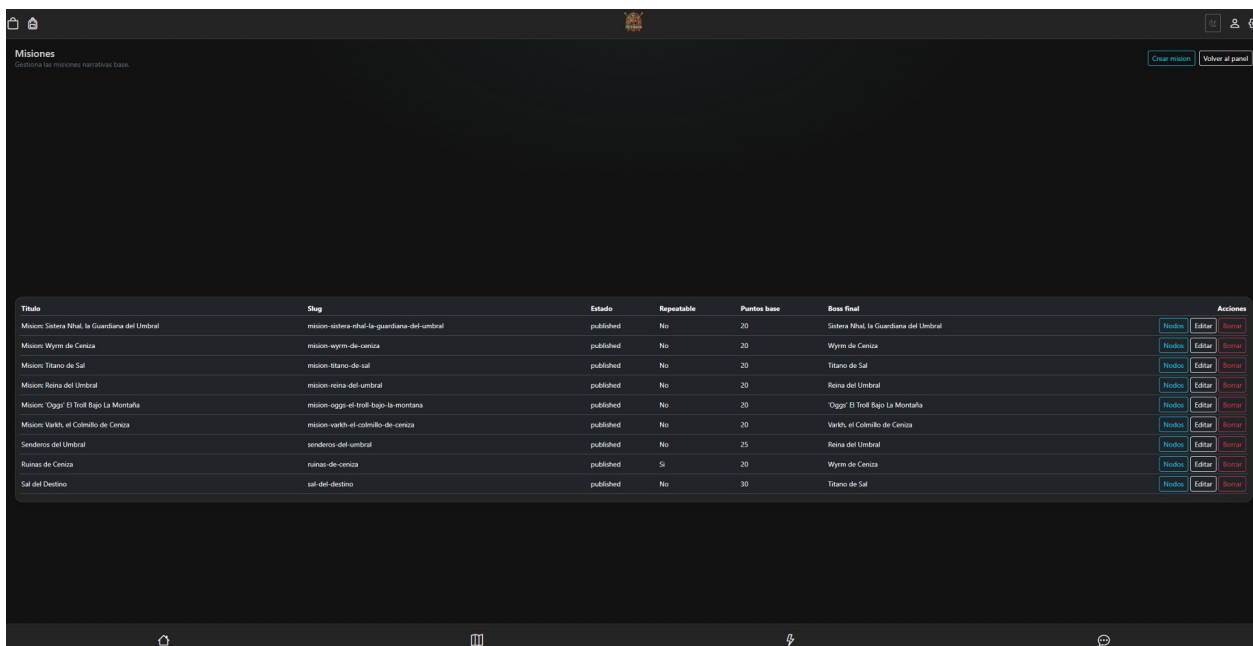
- Entrar al módulo “Misiones”.
- Crear una misión nueva (título, descripción, configuración base).
- Editar una misión existente para ajustar su contenido o parámetros.
- Eliminar una misión cuando se retire del sistema.
- Gestionar su estado (draft/published) según el flujo interno definido.

Validaciones / mensajes

- Campos obligatorios: título y datos mínimos para definir la misión.
- Estado de publicación: solo se permite publicar si cumple la coherencia definida.
- Mensajes de confirmación al crear/editar/eliminar.

Rol que la ve

- Administrador.



Título	Slug	Estado	Repetible	Puntos base	Bos final	Acciones
Misión: Sistema Nhal, la Guardiania del Umbral	misión-sistema-nhal-la-guardiana-del-umbral	published	No	20	Sistema Nhal, la Guardiania del Umbral	Nodos Editar Borrar
Misión: Wyrm de Ceniza	misión-wyrm-de-ceniza	published	No	20	Wyrm de Ceniza	Nodos Editar Borrar
Misión: Titano de Sal	misión-titano-de-sal	published	No	20	Titano de Sal	Nodos Editar Borrar
Misión: Reina del Umbral	misión-reina-del-umbral	published	No	20	Reina del Umbral	Nodos Editar Borrar
Misión: 'Ogg' El Troll Bajo La Montaña	misión-ogg-el-troll-bajo-la-montaña	published	No	20	'Ogg' El Troll Bajo La Montaña	Nodos Editar Borrar
Misión: Varkh, el Colmillo de Ceniza	misión-varkh-el-colmillo-de-ceniza	published	No	20	Varkh, el Colmillo de Ceniza	Nodos Editar Borrar
Senderos del Umbral	senderos-del-umbral	published	No	25	Reina del Umbral	Nodos Editar Borrar
Ruinas de Ceniza	ruinas-de-ceniza	published	Si	20	Wyrm de Ceniza	Nodos Editar Borrar
Sal del Destino	sal-del-destino	published	No	30	Titano de Sal	Nodos Editar Borrar

****Ilustración: CRUD misiones**



9.4 Gestión de nodos (CRUD)

Qué hace

Permite gestionar los nodos narrativos que componen una misión. Cada nodo representa un punto del grafo: texto narrativo, contexto del momento y el conjunto de elecciones posibles desde ese nodo.

Pasos de uso

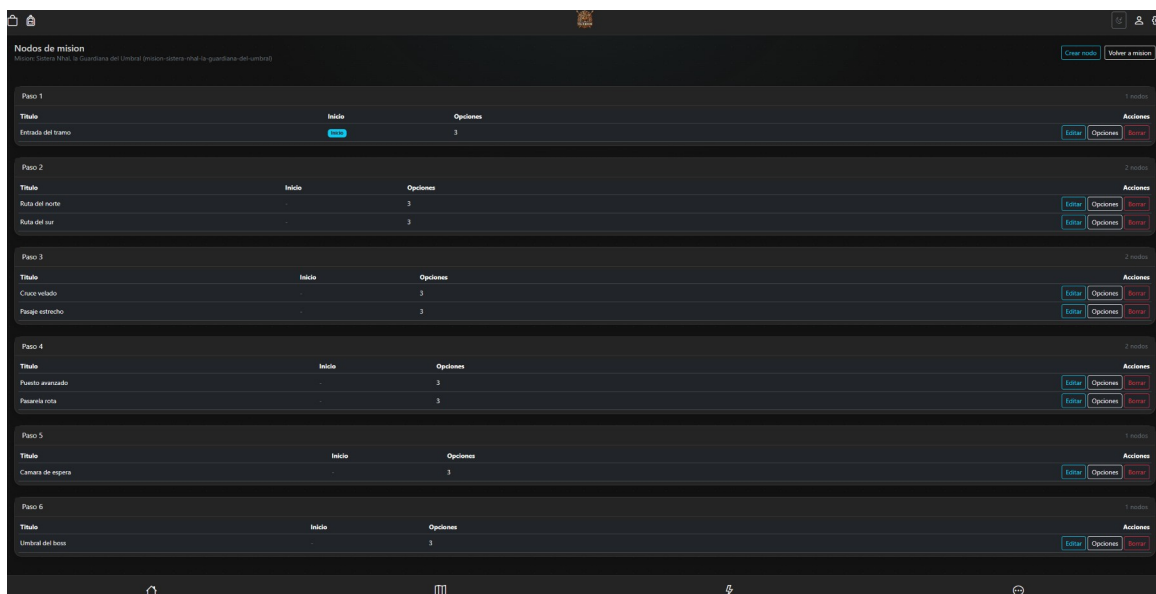
- Entrar al módulo “Nodos” dentro del panel admin (o desde el contexto de una misión).
- Crear un nodo asignándolo a una misión concreta.
- Editar el texto y propiedades del nodo.
- Eliminar nodos que ya no formen parte del recorrido.

Validaciones / mensajes

- Asociación obligatoria a misión.
- Contenido mínimo del nodo: texto y configuración necesaria para ser jugable.
- Avisos de coherencia cuando la eliminación afecte a enlaces existentes en elecciones.

Rol que la ve

- Administrador.



***Ilustración: CRUD nodos*

9.5 Gestión de elecciones (CRUD + enlaces)

Qué hace

Permite crear y editar las elecciones de cada nodo y definir hacia qué nodo conducen. Es la parte que construye el grafo real: cada elección conecta nodos y determina el avance de la misión. También permite asignar dificultad/efecto cuando aplique.

Pasos de uso

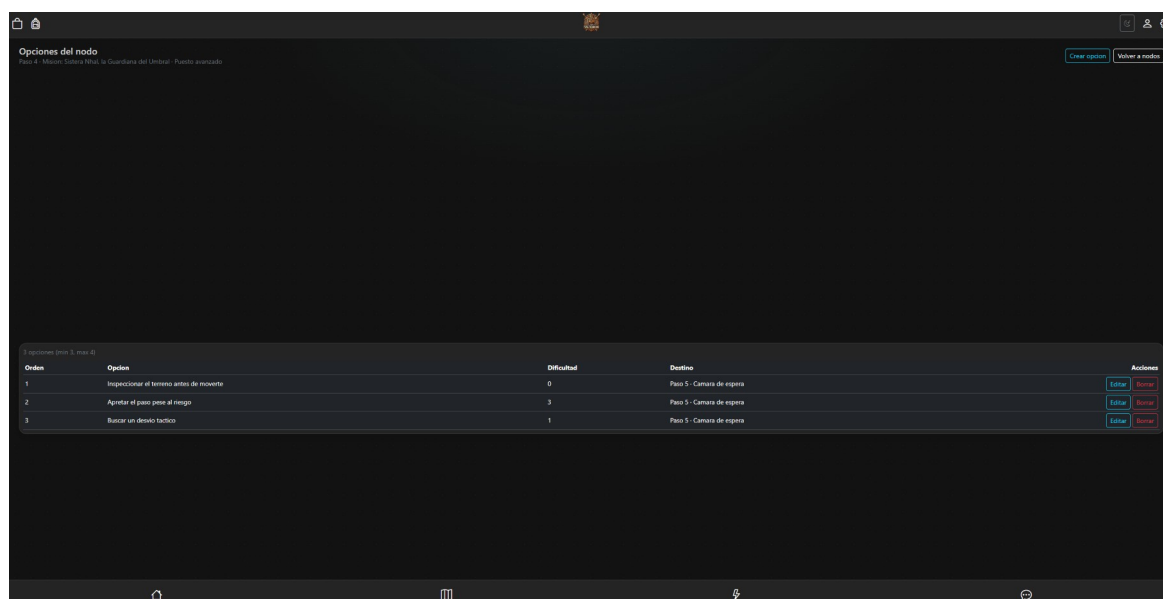
- Entrar al módulo “Elecciones” o gestionarlas desde un nodo concreto.
- Crear una elección dentro de un nodo.
- Definir el texto mostrado al jugador.
- Configurar el enlace al siguiente nodo (o final de misión según estructura definida).
- Editar elecciones para ajustar narrativa, dificultad y enlaces.
- Eliminar elecciones que ya no se utilicen.

Validaciones / mensajes

- Elección vinculada obligatoriamente a un nodo.
- Enlace válido: el siguiente nodo debe existir y pertenecer al contexto correcto.
- Mensajes de aviso si la elección queda sin enlace o rompe el recorrido.

Rol que la ve

- Administrador.



Orden	Opciones	Dificultad	Destino	Acciones
1	Inspeccionar el terreno antes de moverse	0	Paso 1 - Cámara de espera	Editar Borrar
2	Apretar el paso para el riesgo	3	Paso 3 - Cámara de espera	Editar Borrar
3	Buscar un diseño tático	1	Paso 5 - Cámara de espera	Editar Borrar

***Ilustración: CRUD elecciones y enlaces*



9.6 Publicar/Retirar misión (validación grafo)

Qué hace

Permite controlar qué misiones están disponibles para los jugadores. Publicar una misión activa su visibilidad en el listado de misiones del jugador. Retirla la oculta sin necesidad de borrar contenido. Antes de publicar, se ejecuta la validación de coherencia del grafo para evitar misiones rotas (nodos inaccesibles, enlaces inválidos o finales inconsistentes).

Pasos de uso

- Seleccionar una misión desde el módulo de misiones.
- Ejecutar la acción de “Publicar” cuando esté preparada.
- Si se detectan errores en el grafo, corregir nodos/elecciones y repetir la validación.
- Usar la acción de “Retirar” cuando se necesite ocultar la misión a jugadores manteniendo su contenido.

Validaciones / mensajes

- Validación del grafo obligatoria antes de publicar.
- Mensajes claros indicando qué parte del grafo no cumple (nodos sin salida, enlaces inválidos, etc.).
- Confirmación de cambio de estado (publicada/retirada).

Rol que la ve

- Administrador.

***Ilustración: Publicación de misión y validación del grafo*



9.7 Gestión de ítems/tienda (CRUD)

Qué hace

Permite administrar el catálogo de objetos del juego: crear ítems, definir rareza, precio, bonificaciones y tipo (equipable/consumible). Estos ítems se muestran en la tienda y se integran con inventario y equipamiento del jugador.

Pasos de uso

- Entrar al módulo “Ítems” o “Tienda” desde el panel admin.
- Crear un ítem nuevo indicando sus propiedades.
- Editar ítems para ajustar balance, precios o efectos.
- Eliminar ítems retirados del catálogo según reglas del sistema.

Validaciones / mensajes

- Campos obligatorios: nombre, tipo, precio y datos de bonificación cuando aplique.
- Valores numéricos coherentes (precio y stats).
- Avisos si un ítem está vinculado a inventarios o a reglas internas del sistema.

Rol que la ve

- Administrador.

Montura en venta

Semana 7, 2026

Montura de la semana

Selecciona una montura

Guardar montura

Montura actual: Montura Entrenada

***Ilustración: CRUD ítems/tienda*



9.8 Gestión de usuarios/roles/plan

Qué hace

Permite administrar cuentas de usuario y controlar permisos mediante roles (jugador/administrador) y plan cuando aplica. Desde aquí se mantiene la seguridad del sistema, se controlan accesos y se aplican reglas internas como restricciones de razas o privilegios del panel admin.

Pasos de uso

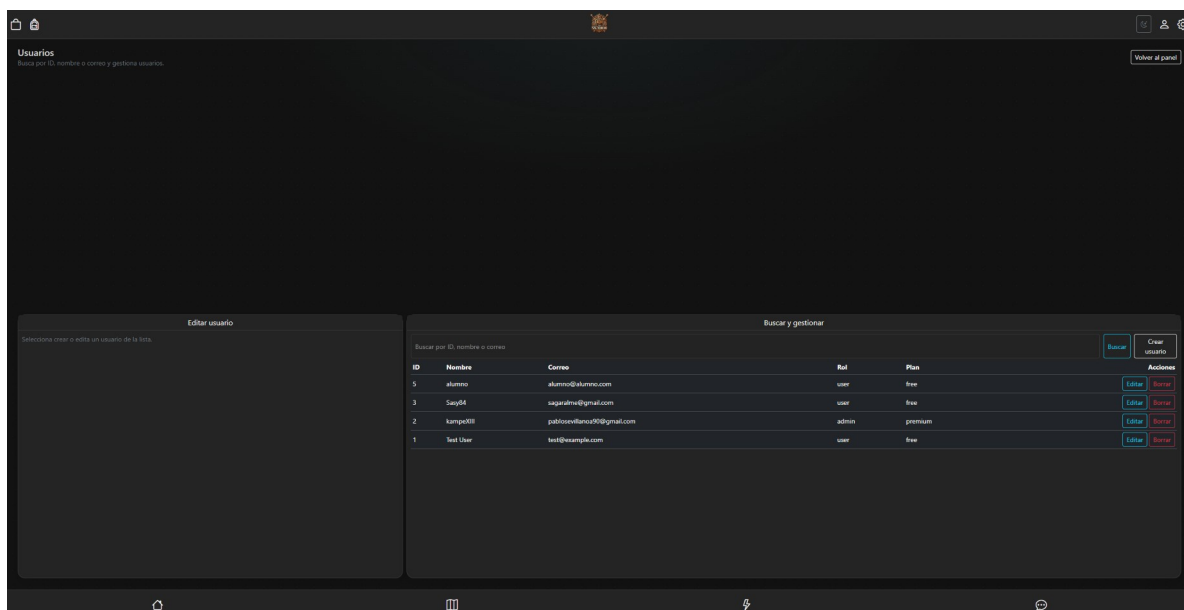
- Entrar al módulo “Usuarios”.
- Consultar listado de usuarios registrados.
- Editar un usuario para cambiar rol o plan cuando sea necesario.
- Revisar estado general del usuario dentro del sistema.

Validaciones / mensajes

- Control estricto de permisos: solo administradores pueden modificar roles/plan.
- Mensajes de confirmación al guardar cambios.
- Reglas de coherencia: el rol afecta a accesos y visibilidad de módulos, y el plan afecta a restricciones internas.

Rol que la ve

- Administrador.



***Ilustración: Gestión de usuarios, roles y plan*



10. Estructura del proyecto

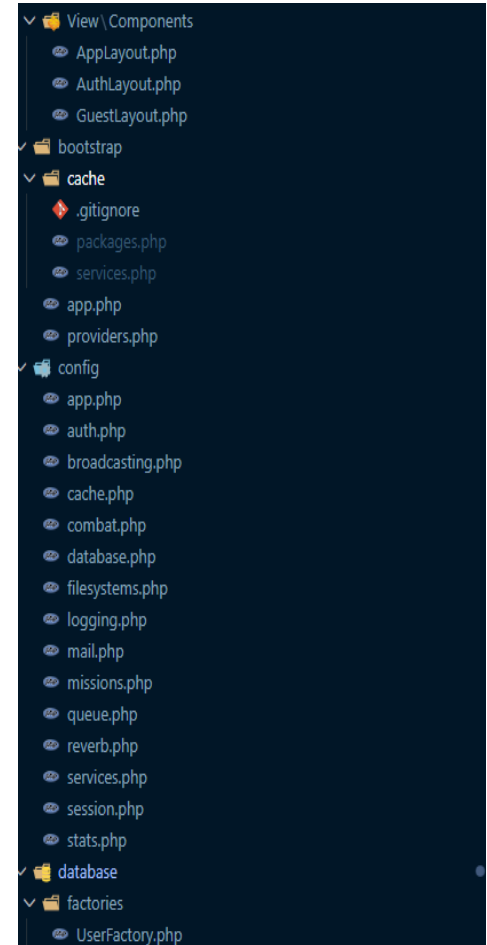
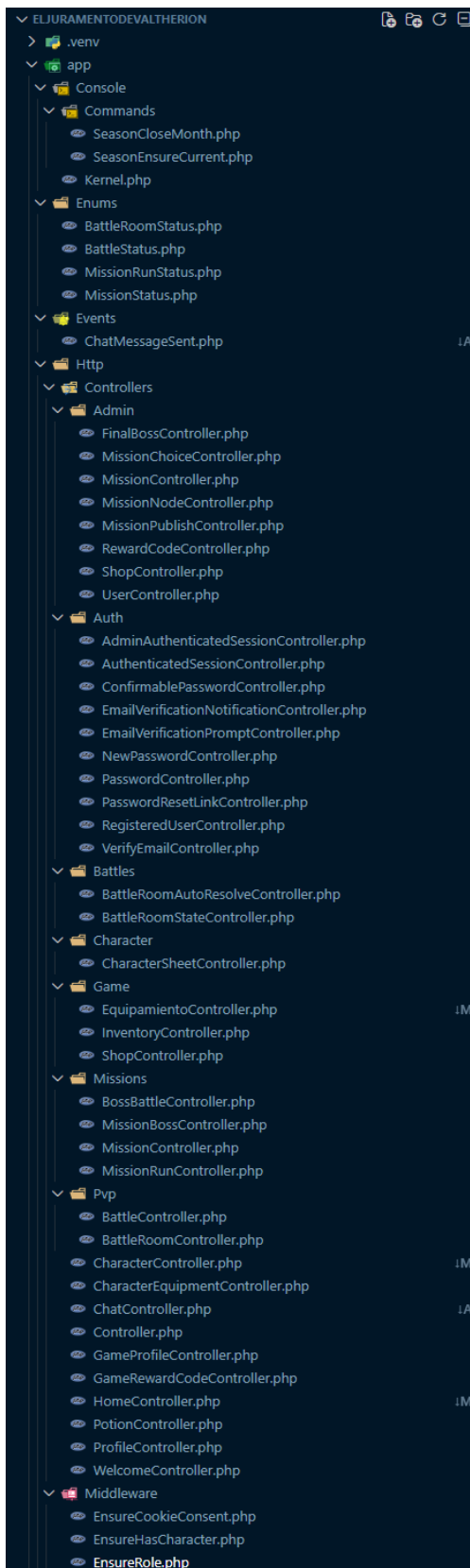
10.1 Árbol de directorios (seleccionado y legible)

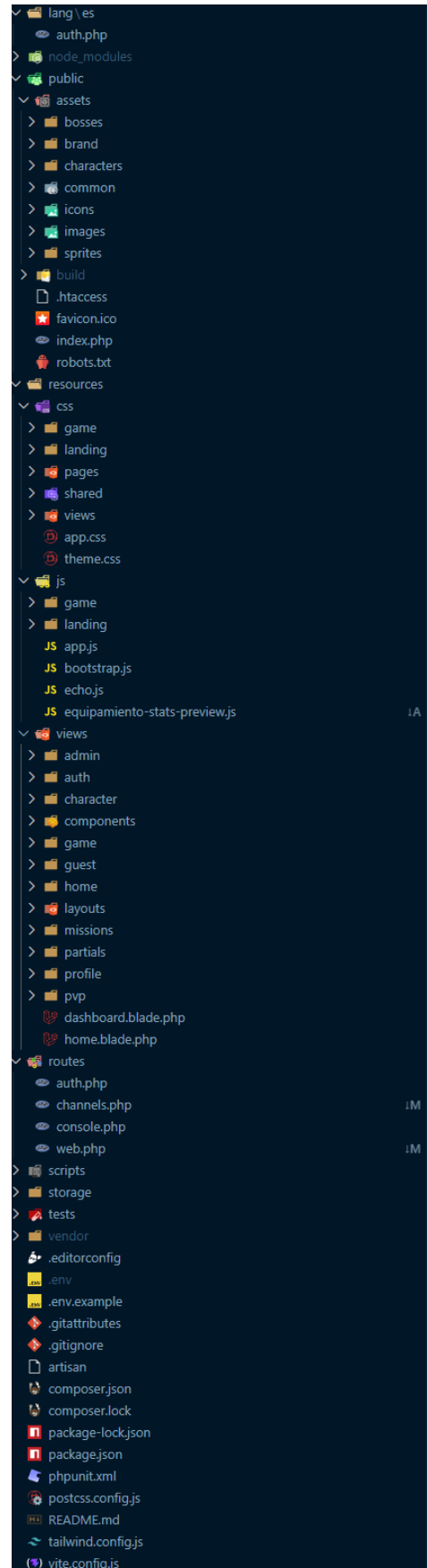
La estructura del proyecto sigue el estándar de Laravel, pero organizada por módulos para que el código sea legible y mantenible. Los directorios principales son:

- **app/**
 - **Console/**
 - **Commands/** (comandos Artisan propios: cierres de temporada, verificaciones, etc.)
 - **Enums/** (estados y constantes tipadas: misión, combate, salas, etc.)
 - **Events/** (eventos para tiempo real, como envío de mensajes)
 - **Http/**
 - **Controllers/**
 - **Admin/** (gestión completa del panel: misiones, nodos, elecciones, bosses, usuarios, tienda...)
 - **Auth/** (login, registro, verify email, reset password...)
 - **Battles/** (estado de combate y resolución automática)
 - **Character/** (ficha, equipamiento, estado del personaje)
 - **Game/** (módulos jugables: inventario, tienda, equipamiento...)
 - **Missions/** (ejecución de misiones, runs, boss final, opciones...)
 - **Pvp/** (combate jugador vs jugador, sala/lobby, etc.)
 - **Middleware/** (control de rol, personaje obligatorio, consentimiento cookies...)
 - **Requests/** (Form Requests por módulo: Admin, Missions, Auth, PvP...)
 - **Models/** (modelos Eloquent del dominio: Character, Mission, Item, Battle, Chat...)
 - **Services/**
 - **Combat/** (motores de combate PvE/PvP y formateo de logs)
 - **Missions/** (validador de grafo, recompensas, ejecución de runs...)
 - **Stats/** (cálculo de estadísticas, multiplicadores, servicios auxiliares)



- **View/Components/** (layouts/componentes Blade reutilizables)
- **bootstrap/, config/** (configuración del framework y módulos: session, database, reverb, missions, stats...)
- **database/**
 - **migrations/** (estructura completa del dominio: razas, ítems, equipo, misiones, combate, chat, temporadas...)
 - **seeders/** (datos base: admin, razas, tienda, misiones, bosses...)
 - **database.sqlite** (base de datos portable del proyecto)
- **public/**
 - **assets/** (recursos del juego: bosses, sprites, icons, imágenes...)
 - **build/** (assets compilados por Vite para evaluación/producción local)
 - **index.php, robots.txt, favicon.ico, .htaccess**
- **resources/**
 - **css/** (separación por zonas: game, landing, pages, shared + app.css/theme.css)
 - **js/** (scripts por zonas + echo/reverb)
 - **views/** (vistas Blade por módulos: admin, auth, game, missions, character, pvp, layouts, components...)
- **routes/**
 - **web.php** (rutas principales del sistema)
 - **auth.php** (rutas de autenticación)
 - **channels.php** (canales de broadcasting)
 - **console.php**
- **storage/, tests/, vendor/** (estructura estándar de Laravel)
- Archivos raíz: **.env(.example), composer.json/lock, package.json/lock, tailwind.config.js, vite.config.js, README.md**





***Ilustración Árbol del proyecto en VS Code*

10.2 Explicación por carpetas

app/

Es el núcleo del proyecto. Aquí vive la lógica real: controladores, servicios, modelos, validaciones y middleware. La clave es la separación por responsabilidad:

- **Http/Controllers/** está dividido por áreas reales del sistema (Admin, Missions, Game, Character, PvP...), lo que evita controladores “gigantes” y hace que cada módulo tenga su propio bloque.
- **Http/Requests/** contiene la validación de formularios por caso de uso (crear misión, publicar, elegir opción, iniciar run, etc.). Esto mantiene los controladores limpios y con reglas centralizadas.
- **Services/** encapsula la lógica compleja (combate, validación del grafo de misiones, cálculo de stats, recompensas). Así se evita mezclar reglas de negocio con capas de presentación.
- **Models/** representa el dominio del juego: personaje, inventario, equipo, misiones, combate, chat, temporadas... con relaciones Eloquent consistentes.
- **Middleware/** protege el flujo: rol requerido, personaje obligatorio, control de acceso y comportamiento transversal.
- **Enums/** permite manejar estados del sistema con consistencia (por ejemplo, estados de combate o misiones), evitando “strings mágicos”.
- **Events/** soporta el tiempo real (chat) cuando el servicio está activo.

database/

Contiene toda la trazabilidad estructural del sistema:

- **migrations/** definen el modelo relacional del proyecto (tablas, pivotes, restricciones).
- **seeders/** generan un estado inicial jugable y evaluable (razas, tienda, misiones, bosses, admin...).
- **database.sqlite** permite que el proyecto sea portable y ejecutable sin montar un servidor MySQL.

resources/

Es la capa de interfaz y assets no públicos directos:

- CSS y JS están separados por áreas (game/landing/shared), lo que facilita mantener una UI tipo aplicación (pantallas por secciones, consistencia visual, responsive).



- **views/** está estructurado por módulos reales (admin/auth/game/missions/character/pvp), lo que encaja con la organización de controladores y rutas.

public/

Es el punto de salida web:

- **assets/** contiene los recursos gráficos del juego (sprites, imágenes, iconos, bosses...).
- **build/** contiene el resultado compilado por Vite, necesario para ejecutar el proyecto en modo evaluación sin depender de npm run dev.

routes/

Centraliza cómo se accede a cada zona:

- **web.php** organiza el flujo del sistema (público, juego, admin) con middleware.
- **auth.php** mantiene la autenticación separada y limpia.
- **channels.php** define canales si el chat/broadcasting está activo.

config/

Incluye configuración del framework y módulos propios (por ejemplo, reverb, sesiones, stats, misiones, combate). Esto permite ajustar comportamiento sin tocar lógica de negocio.

11. Descripción técnica

11.1 Arquitectura MVC en Laravel

La aplicación está construida sobre Laravel 12 siguiendo el patrón MVC, separando responsabilidades para mantener el proyecto escalable y fácil de mantener.

El flujo general es: la petición entra por una ruta (routes), pasa por middleware (seguridad/condiciones), llega a un controlador (orquesta el caso de uso), se apoya en modelos y servicios (reglas de negocio y datos) y finalmente devuelve una vista Blade (UI) o una respuesta de redirección con estado.

En este proyecto, el enfoque MVC se refuerza con dos capas clave:

- Form Requests para validación y autorización previa.
- Services para encapsular lógica compleja (misiones, combate, stats, recompensas), evitando que el controlador se convierta en “lógica mezclada”.

11.2 Rutas (web.php / auth.php)

Las rutas están organizadas para reflejar las zonas reales del sistema:

- Zona pública (home/lore, legales, acceso a auth).
- Zona privada del juego (requiere autenticación y, cuando corresponde, personaje creado).
- Zona admin (requiere autenticación y rol administrador).
- Funcionalidades especiales (chat/broadcasting si el servicio está activo).

En términos prácticos:

- routes/web.php centraliza la navegación principal, agrupando rutas por middleware y prefijos coherentes.
- routes/auth.php contiene el bloque de autenticación (registro, login, verify email, reset password...) generado por el starter kit y adaptado al flujo del proyecto.
- routes/channels.php define los canales de broadcasting cuando el chat está activo.

11.3 Controladores (por módulos: Game, Missions, Admin, PvP, Chat)

Los controladores se agrupan por módulo para que cada zona tenga su propio “paquete” de lógica de entrada:

- **Auth:** sesión, registro, verificación y recuperación de contraseña.
- **Game:** home del juego y módulos principales (inventario, tienda, equipamiento...).
- **Character:** ficha del personaje, estadísticas, y lógica asociada a su estado.



- **Missions:** ejecución de misiones por nodos, selección de opciones, runs, cierre con boss y recompensas.
- **Battles:** estado del combate, turnos, acciones, resolución automática cuando corresponde.
- **PvP:** sala/lobby, combate PvP y gestión de estado competitivo.
- **Admin:** CRUD completo (misiones, nodos, elecciones, bosses, tienda, usuarios) y lógica de publicación/validación.

La intención es que el controlador orqueste, pero que la lógica pesada viva en services y modelos.

11.4 Modelos y relaciones (1:1, 1:N, N:M)

El dominio del juego se modela con Eloquent, manteniendo relaciones coherentes para garantizar integridad y evitar inconsistencias.

Relaciones típicas del proyecto:

- **1:1:** Usuario ↔ Personaje (un usuario no puede tener dos personajes).
- **1:N:** Misión → Nodos, Nodo → Elecciones, Personaje → Runs/historial.
- **N:M:** Personaje ↔ Ítems (inventario mediante tabla pivote con cantidades/estado), y relaciones similares donde haya asociaciones múltiples.

Además, el sistema incluye modelos específicos para módulos avanzados:

- Combate (batallas, turnos, estados).
- Chat (salas y mensajes).
- Temporadas/ranking .

11.5 Migraciones y seeders

La base de datos se construye completamente con migraciones, permitiendo recrear el proyecto desde cero en cualquier entorno local de evaluación.

Las migraciones definen:

- Tablas principales del dominio (razas, personajes, ítems, misiones, bosses, combate, chat...).
- Tablas pivote (inventario/equipamiento y relaciones N:M).
- Restricciones (PK/FK, unicidades, campos obligatorios, integridad entre nodos/opciones cuando corresponde).

Los seeders se usan para dejar el proyecto evaluable sin trabajo manual:

- Usuario administrador inicial.
- Datos base de razas, tienda, misiones y bosses.
- Contenido mínimo suficiente para probar el ciclo jugable completo.

11.6 Validación (Form Requests, mensajes, CSRF)

La validación se gestiona con Form Requests en los módulos donde hay operaciones críticas:

- Admin (crear/editar bosses, misiones, nodos, elecciones, ítems).
- Missions (iniciar run, elegir opción, acciones del boss, abandono).
- Auth (login/registro según el flujo).

Esto permite:

- Mantener reglas centralizadas y reutilizables.
- Controlar autorización (por rol) desde la propia request cuando conviene.
- Unificar mensajes y garantizar consistencia en la experiencia.

Además, todas las peticiones POST/PUT/DELETE están protegidas con CSRF, siguiendo el estándar de Laravel.

11.7 Seguridad (auth, roles, middleware, sesiones/cookies, CSRF)

La seguridad se basa en varias capas:

- **Autenticación:** rutas privadas protegidas, sesión controlada por Laravel.
- **Roles y permisos:** middleware que restringe acceso a /admin y operaciones sensibles.
- **Control de estado jugable:** middleware que fuerza tener personaje creado antes de acceder a módulos del juego.
- **CSRF:** protección integrada en formularios.
- **Sesiones/cookies:** gestión estándar de Laravel, con configuración alineada a ejecución local y a requisitos del módulo.

11.8 Tiempo real (Chat: eventos, broadcasting, Reverb)

El chat funciona como módulo interno de comunicación usando broadcasting en Laravel:

- Eventos del lado servidor para emisión de mensajes.
- Canales definidos para separar salas o contextos.
- Cliente escuchando eventos en tiempo real.



Cuando el servicio está activo, Reverb se encarga de la capa WebSocket, permitiendo comunicación instantánea sin recargar la página.

11.9 Comandos artisan usados (migrate, seed, build, limpieza, tiempo real)

Durante desarrollo y evaluación se utilizan comandos estándar para asegurar reproducibilidad:

- `php artisan serve` (ejecución local)
- `php artisan migrate` / `php artisan migrate:fresh --seed` (reconstrucción completa)
- `php artisan optimize:clear` (limpieza de cachés para evitar inconsistencias)
- `php artisan reverb:start` (arranque del servicio de tiempo real cuando se prueba el chat)
- Otros comandos auxiliares (creación de controladores, requests, migraciones, seeders, etc.)

12. Conclusión

12.1 Objetivos alcanzados

El Juramento de Valtherion cumple el objetivo principal del proyecto: desarrollar una aplicación web completa en entorno servidor que funcione como RPG desde navegador, con un ciclo jugable coherente y un sistema de administración capaz de mantener y ampliar el contenido sin depender de modificaciones constantes en la base de datos. La aplicación integra autenticación, control de acceso, creación de personaje y reglas de raza, progresión mediante estadísticas y recursos, inventario y equipamiento con impacto real sobre el personaje, tienda funcional y un sistema de misiones ramificadas basado en nodos y elecciones que culmina en combate PvE contra jefes finales con recompensas. De forma complementaria, se han incorporado módulos sociales y competitivos (chat en tiempo real cuando el servicio está activo y PvP) para reforzar la experiencia global.

A nivel técnico, el proyecto se ha desarrollado con Laravel 12 siguiendo MVC, con una organización clara por módulos y una separación real entre controladores, modelos y servicios. La persistencia se resuelve con SQLite como fichero portable, lo que facilita la evaluación y asegura que el proyecto se puede ejecutar y reconstruir en local mediante migraciones y seeders. Además, la interfaz se ha maquetado con Blade y Tailwind, manteniendo un enfoque tipo aplicación web responsive para móvil, tablet y escritorio.

12.2 Dificultades y soluciones

Durante el desarrollo, uno de los retos principales fue garantizar coherencia e integridad entre módulos que dependen entre sí. El sistema de misiones ramificadas exige que la estructura de nodos y elecciones sea consistente y que no existan enlaces inválidos que rompan el flujo jugable. Para resolverlo se aplicaron validaciones y reglas de consistencia antes de permitir ciertos estados (especialmente en publicación y uso desde la zona de jugador), evitando que contenido incompleto llegue a ejecución real.

Otro punto crítico fue mantener la lógica compleja fuera de los controladores. Módulos como combate, recompensas, cálculo de estadísticas y progresión tienden a crecer rápidamente en complejidad si se mezclan en el mismo lugar. La solución fue estructurar la lógica en servicios dedicados y apoyar el control de entradas con Form Requests, manteniendo trazabilidad y control de errores.

También se afrontó el reto de reproducibilidad en entorno de evaluación. Para que el proyecto pueda revisarse sin depender del autor, se trabajó con migraciones, seeders y comandos de limpieza, además de una organización de carpetas clara y una planificación documentada mediante bitácora diaria. Esto garantiza que el estado inicial del juego y el contenido mínimo necesario pueden regenerarse desde cero.



12.3 Mejoras futuras

Como mejora futura, se puede ampliar la cantidad de contenido jugable (más misiones, rutas ramificadas, bosses y variedad de ítems) aprovechando que la base del sistema ya es ampliable desde administración. También sería posible enriquecer el módulo PvP con emparejamiento más avanzado, rankings y temporadas más detalladas, además de reforzar el chat con herramientas de moderación y presencia en salas. A nivel técnico, podría incorporarse una capa adicional de optimización y monitorización si el proyecto se llevara a un entorno de producción real.

13. Webgrafía

La documentación y referencias consultadas durante el desarrollo de El Juramento de Valtherion se apoyan principalmente en fuentes oficiales del framework y en estándares reconocidos para frontend, seguridad y accesibilidad. Estas referencias se han utilizado para resolver dudas técnicas, validar buenas prácticas y asegurar que la aplicación se ajusta a un enfoque correcto de desarrollo web en entorno servidor.

- Laravel — Documentación oficial (framework, estructura, despliegue, configuración)
<https://laravel.com/docs>
- Laravel Learn — Recursos oficiales de aprendizaje
<https://laravel.com/learn>
- Laravel Breeze — Starter Kit de autenticación
<https://laravel.com/docs/starter-kits#laravel-breeze>
- Laravel — Eloquent ORM (relaciones, pivotes, consultas)
<https://laravel.com/docs/eloquent>
- Laravel — Broadcasting (eventos y tiempo real)
<https://laravel.com/docs/broadcasting>
- Laravel Reverb — WebSockets / tiempo real ()
<https://laravel.com/docs/reverb>
- Laravel — Vite (gestión y compilación de assets)
<https://laravel.com/docs/vite>
- SQLite — Documentación oficial (motor embebido, fichero portable)
<https://www.sqlite.org/docs.html>
- MDN Web Docs — Referencia HTML, CSS y JavaScript
<https://developer.mozilla.org/>
- OWASP — Buenas prácticas y guías de seguridad web
<https://owasp.org/>
- WCAG (W3C) — Estándar de accesibilidad web
<https://www.w3.org/WAI/standards-guidelines/wcag/>